CARTEIRA DIGITAL

Documentação da Arquitetura de Software — Entrega 3

Template arc42 versão 8.2-PT · C4 Model (contexto, contêineres e componentes) · UML · Architecture Inception Canvas

Aplicação distribuída em nuvem pública: Microfrontend (SPA) · API Gateway (AWS) · Backend for Frontend · Microsserviços com Database per Service (MongoDB Atlas e Azure SQL) · Azure Functions · Arquitetura Orientada a Eventos (Azure Service Bus) · Clean Architecture e Vertical Slice

INTEGRANTES DO GRUPO

Gabriel João Conti · RA 40095589

Guilherme Celli Fadel · RA 40101681

Lucas Arthur da Silva Reali · RA 40092691

Thiago Akimoto · RA 40090610

Arquitetura de Soluções com Cloud · PJBL

Bacharelado em Engenharia de Software · PUCPR

Novembro de 2026

Sumário

Links da entrega
Índice de Diagramas e Prints
Como ler esta entrega
1. Introdução e Objetivos
2. Restrições Arquiteturais
3. Contexto e Escopo
4. Estratégia de Solução
5. Visão de Blocos de Construção
6. Visão de Tempo de Execução
7. Visão de Implantação
8. Conceitos Transversais
9. Decisões Arquiteturais
10. Requisitos de Qualidade
11. Riscos e Débitos Técnicos
12. Glossário
Apêndice A. Prints da solução em nuvem
Referências

Links da entrega

Todos os endereços abaixo são públicos e estão em nuvem; nenhum depende de `localhost`. O código está num único repositório (monorepo pnpm), com uma pasta por projeto.

Item	Link
GitHub (todos os projetos)	github.com/lucasreali/digital-wallet-monorepo Microfrontend: `apps/web` · BFF: `apps/bff` · Microsserviço MongoDB: `services/svc-identidade` · Microsserviço SQL: `services/svc-carteira` · Azure Function: `services/fn-notificacoes`
Docker Hub (imagens)	hub.docker.com/u/f4d3l `f4d3l/carteira-web:v4` · `f4d3l/carteira-bff:v1` · `f4d3l/carteira-svc-identidade:v2` · `f4d3l/carteira-svc-carteira:v2` · `f4d3l/carteira-fn-notificacoes:v2` (esta é de referência: na nuvem a Function roda no Azure Functions)
Swagger — BFF (pelo API Gateway)	i12uxusdx6.execute-api.us-east-1.amazonaws.com/docs
Swagger — svc-identidade (MongoDB)	svc-identidade.wonderfuldune-fe1dc2f4.canadacentral.azurecontainerapps.io/docs
Swagger — svc-carteira (Azure SQL)	svc-carteira.wonderfuldune-fe1dc2f4.canadacentral.azurecontainerapps.io/docs
Swagger — Eventos (AsyncAPI 3.0)	github.com/lucasreali/digital-wallet-monorepo/blob/main/docs/events/asyncapi.yaml Documentação renderizada: AsyncAPI Studio com o arquivo do repositório
Frontend	web.wonderfuldune-fe1dc2f4.canadacentral.azurecontainerapps.io
API Gateway (AWS)	i12uxusdx6.execute-api.us-east-1.amazonaws.com/aggregated-data A raiz não tem rota: o gateway repassa qualquer caminho ao BFF (ex.: `/users`, `/wallets`, `/docs`).
Azure Function	…azurewebsites.net/api/insights (score) · …/api/activity (feed)
Vídeo no YouTube (não listado)	**[preencher com o link após a publicação]**

Índice de Diagramas e Prints

Os diagramas seguem o mapeamento entre arc42 e C4 Model usado na disciplina: contexto na seção 3, contêineres e componentes na seção 5, sequências UML na seção 6 e implantação na seção 7. Todos foram feitos no draw.io (arquivo `carteira-digital-diagramas-entrega3.drawio`), gerados por scripts a partir do conteúdo desta documentação.

Figura	Diagrama	Notação	Seção
1	Architecture Inception Canvas	Canvas oficial do arc42	1.4
2	Contexto do sistema (Nível 1)	C4 Model	3.1
3	Contêineres (Nível 2)	C4 Model	5.1
4 a 7	Componentes do BFF, do svc-identidade, do svc-carteira e da fn-notificacoes (Nível 3)	C4 Model	5.2 a 5.5
8 e 9	Sequências: depósito de ponta a ponta e GET /aggregated-data	UML 2.5.1	6.1 e 6.2

10	Implantação em nuvem pública	C4 Model (implantação)	7.1
11	Modelo de dados, um banco por serviço	Entidade-Relacionamento, pé de galinha	8.1

Os prints exigidos (Swagger do BFF, Swagger dos microsserviços, frontend funcionando e API Gateway) estão no Apêndice A, junto com prints do MongoDB Atlas, do Azure SQL, do Docker Hub e dos Container Apps.

Como ler esta entrega

> **O que mudou desde a Entrega 2.** A Entrega 2 projetou uma carteira completa (PIX, cartões, KYC, seis microsserviços em Java/Spring, PostgreSQL e Azure API Management). O enunciado da Entrega 3 fixa outro recorte: um frontend SPA, um BFF em Node.js com `GET /aggregated-data`, um microsserviço no MongoDB Atlas, outro no Azure SQL, uma Azure Function, eventos e o API Gateway da AWS. Este documento descreve **o sistema que foi construído e está no ar**: dois microsserviços (Identidade e Carteira), o BFF, a Function e o frontend, todos em TypeScript. Os estilos da Entrega 2 continuam (BFF, Database per Service, eventos, serverless, Clean Architecture e Vertical Slice); o que mudou foi o escopo e a tecnologia, como explica a seção 4.3.
>
> **Fonte da verdade.** Cada afirmação sobre comportamento foi conferida no código do repositório (commit `b3eb2d8`) e nas URLs públicas. Limitações conhecidas não foram escondidas: estão na seção 11.

1. Introdução e Objetivos

A Carteira Digital ("Lastro", no frontend) é uma aplicação distribuída em nuvem pública para gerenciar pessoas e suas carteiras. Um operador cadastra pessoas, abre uma carteira para cada uma, registra depósitos e saques, consulta o extrato e acompanha um painel com o score financeiro e o feed de atividade. Cada operação relevante vira um evento, e uma Azure Function usa esses eventos para montar o feed e calcular o score.

O objetivo da entrega é demonstrar, com URLs de nuvem, a combinação de API Gateway, BFF, microsserviços com bancos próprios, serverless e eventos pedida no PJBL, com o código organizado em Clean Architecture e Vertical Slice e verificado por testes unitários e de arquitetura.

1.1 Visão Geral dos Requisitos

Funcionalidade	Regras de negócio	Contêiner responsável
Usuários: cadastrar, listar, consultar, editar e excluir	E-mail válido e único (409 se já existir); não exclui quem ainda tem carteira (409)	svc-identidade (MongoDB Atlas)
Carteiras: abrir, listar, consultar e encerrar	Uma carteira por usuário (409); nasce com saldo R$ 0,00; só encerra com saldo zero (409)	svc-carteira (Azure SQL)
Depósito, saque e extrato	Valor maior que zero e com no máximo 2 casas (422 no svc-carteira; pelo BFF, valor menor que 0,01 já volta 400); saque não deixa saldo negativo (422); extrato do mais novo ao mais antigo, com o saldo após cada lançamento	svc-carteira (Azure SQL)
Feed de atividade	Últimos N eventos (1 a 100) de usuários e carteiras	fn-notificacoes (Azure Function + Table Storage)
Score financeiro	`round(1000 × depósitos ÷ (depósitos + saques))`, de 0 a 1000; 500 quando não há movimento	fn-notificacoes
Painel agregado	`GET /aggregated-data` devolve usuários, carteiras e score num único JSON	BFF

1.2 Objetivos de Qualidade

#	Objetivo	O que significa neste sistema	Como é verificado
1	Manutenibilidade	Cada caso de uso é uma fatia isolada; o domínio não depende de framework nem de banco	`architecture.spec.ts` em cada backend falha se uma regra de camada for quebrada
2	Integridade do saldo	O saldo sempre bate com os lançamentos, nunca fica negativo e não se perde em gravações simultâneas	Valores em centavos e gravação atômica com conferência do saldo lido (409 em conflito); os testes de depósito e saque cobrem as regras de valor e saldo
3	Implantabilidade	Cada contêiner é publicado e trocado sozinho; os backends são configurados só por variáveis de ambiente	Imagens versionadas no Docker Hub; a mesma imagem de cada backend roda no docker compose e no Azure Container Apps (o frontend embute a URL do gateway no build)
4	Interoperabilidade	Contratos explícitos entre as partes	OpenAPI em cada serviço NestJS (Swagger), cliente do frontend gerado a partir do OpenAPI do BFF e AsyncAPI para os eventos

1.3 Partes Interessadas

Papel	Quem é	Expectativa em relação à arquitetura
Operador da carteira	Usuário do frontend	Telas rápidas, mensagens de erro claras (e-mail repetido, saldo insuficiente) e um painel que reúne tudo
Equipe de desenvolvimento	Lucas (Azure SQL, API Gateway e frontend), Thiago (MongoDB Atlas e svc-identidade), Gabriel (Azure Function e Service Bus) e Guilherme (Docker Hub e Container Apps)	Partes independentes, para cada integrante evoluir e publicar a sua sem quebrar as outras
Professor da disciplina	Avaliador da entrega	Requisitos do enunciado atendidos, URLs de nuvem, arc42, C4 Model, Canvas e prints

1.4 Software Architecture Canvas

A Figura 1 foi preenchida sobre o template oficial do **Architecture Inception Canvas** (canvas.arc42.org, licença CC BY-SA 4.0), nome atual do Software Architecture Canvas. Ele resume em uma página o caso de negócio, a visão funcional, o contexto, as três metas de qualidade principais, as restrições, as hipóteses arquiteturais (detalhadas como ADRs na seção 9) e os riscos (seção 11). Em relação à Entrega 2, todas as caixas foram atualizadas para o sistema implementado.

1.4 Software Architecture Canvas

Architecture Inception Canvas

System:	Created by:	Created for:	Date / Iteration:
Carteira Digital	G. J. Conti · G. C. Fadel · L. A. S. Reali · T. Akimoto	Arquitetura de Soluções com Cloud	Novembro de 2026 · Entrega 3

Business Case

Brief description of the business case or economic driver behind the software system.

- Demonstrar, em nuvem pública, uma carteira digital distribuída: cadastro de pessoas, abertura de carteira, depósitos, saques, extrato e um score financeiro calculado a partir dos eventos, conforme o enunciado do PJBL (Entrega 3).

Functional Overview

The most important functional requirements at a high level.

- Usuários: CRUD completo com e-mail único (svc-identidade, MongoDB Atlas)
- Carteiras: abrir (uma por usuário), consultar e encerrar só com saldo zero (svc-carteira, Azure SQL)
- Depósito, saque e extrato, sem saldo negativo
- Feed de atividade e score de 0 a 1000 (Azure Function alimentada por eventos)
- Painel com GET /aggregated-data: MongoDB + SQL + Function numa resposta

Business Context

Separate your system under construction as a black box from all its communication partners. Communication partners are neighbouring external systems and users.

- **Sistema em escopo**
  Carteira Digital
- **Usuário**
  Operador da carteira, pelo frontend web
- **Sistemas vizinhos**
  Docker Hub (imagens) e GitHub (código e documentação)
- **Provedores de nuvem**
  AWS (API Gateway), Microsoft Azure (Container Apps, Functions, Service Bus, SQL e Storage) e MongoDB Atlas

Organisational Constraints

Any organisational requirement that limits the software architects freedom of decision.

- RO-1 Equipe de 4 integrantes, cada um responsável por uma parte e presente no vídeo
- RO-2 Prazo da Entrega 3 do PJBL
- RO-3 Documentação em arc42, com C4 Model, Canvas e prints
- RO-4 Recursos de nuvem divididos entre contas: AWS, Azure e MongoDB Atlas

Quality Goals

The three most important quality goals for the architecture, which have the highest priority for the most important stakeholder.

- **1. Manutenibilidade**: camadas e fatias verificadas por testes de arquitetura
- **2. Integridade do saldo**: valores em centavos e gravação atômica com controle de concorrência
- **3. Implantabilidade**: imagens no Docker Hub, configuração só por variáveis de ambiente, tudo em nuvem pública

Technical Constraints

Any technical requirement that restricts the software architects freedom of decision

- RT-1 Frontend React (SPA) que só conversa com o BFF
- RT-2 BFF em Node.js com GET /aggregated-data
- RT-3 Microsserviço 1 no MongoDB Atlas (Free Tier)
- RT-4 Microsserviço 2 no Azure SQL Database
- RT-5 Azure Function com gatilho HTTP e de mensagem
- RT-6 AWS API Gateway na entrada
- RT-7 Clean Architecture, Vertical Slice, testes unitários e de arquitetura
- RT-8 Imagens no Docker Hub e código público no GitHub
- RT-9 Só URLs de nuvem na demonstração; orçamento acadêmico

Architectural Hypotheses

Resulting architectural hypotheses and important, expensive, large-scale or risky decisions, including justifications.

- ADR-01 Monorepo pnpm com cinco pacotes implantados separadamente
- ADR-02 NestJS + Fastify nos três backends, com o mesmo layout de camadas e fatias
- ADR-03 Database per Service: MongoDB para identidade, Azure SQL para carteira
- ADR-04 Eventos numa fila do Azure Service Bus consumida pela Function
- ADR-05 BFF com contrato OpenAPI e cliente gerado no frontend (Kubb)
- ADR-06 Contêineres no Azure Container Apps; HTTP API do AWS API Gateway na frente do BFF
- ADR-07 Saldo em centavos com controle otimista de concorrência

Technical Challenges & Risks

Identified current known challenges and technical risks

- R-1 Sem autenticação nem limite de requisições; o BFF também responde direto pela URL do Container App (CORS do gateway já restrito ao frontend)
- R-2 Sem outbox: um evento pode se perder se o Service Bus falhar
- R-3 /aggregated-data responde 502 inteiro se uma fonte cair
- R-4 Partida a frio nas camadas gratuitas (gateway corta em 30 s)
- R-5 Abrir carteira não confere se o usuário existe
- R-6 Excluir usuário depende do svc-carteira no ar

Software Architecture Canvas, © 2023 by Gernot Starke, Patrick Roos and arc42 Contributors is licensed under Attribution-ShareAlike 4.0 International.

https://canvas.arc42.org

Figura 1 — Architecture Inception Canvas da Carteira Digital (template oficial arc42, canvas.arc42.org)

2. Restrições Arquiteturais

2.1 Restrições Técnicas

#	Restrição	Origem	Como foi atendida
RT-1	Microfrontend SPA em React ou Angular que consome apenas o BFF	Enunciado	React 19 com TanStack Start; o único endereço configurado é `VITE_BFF_URL`, apontando para o API Gateway
RT-2	BFF em Node.js com proxy do CRUD e `GET /aggregated-data`	Enunciado	NestJS 12 sobre Fastify, 14 rotas
RT-3	Microsserviço 1 no MongoDB Atlas (Free Tier), CRUD e Swagger	Enunciado	svc-identidade com Mongoose 9, banco `identidade`
RT-4	Microsserviço 2 no Azure SQL Database (camada gratuita), CRUD e Swagger	Enunciado	svc-carteira com TypeORM 1.1 e mssql, banco `svc-carteira-db`
RT-5	Azure Function com gatilho HTTP ou de mensagem, exposta por HTTP	Enunciado	fn-notificacoes: um gatilho de Service Bus e dois HTTP
RT-6	AWS API Gateway na entrada	Enunciado	HTTP API `carteira-gateway` em us-east-1, com CORS restrito ao frontend
RT-7	Clean Architecture, Vertical Slice, testes unitários e de arquitetura	Enunciado	Mesmo layout nos três backends; 117 testes em 34 arquivos
RT-8	Imagens no Docker Hub e repositórios públicos no GitHub	Enunciado	Cinco imagens em `f4d3l/carteira-*`; monorepo público
RT-9	Demonstração só com URLs de nuvem e orçamento acadêmico	Enunciado e equipe	Camadas gratuitas ou de consumo; escala a zero onde possível

2.2 Restrições Organizacionais

- **RO-1** Equipe fixa de quatro integrantes; o vídeo precisa mostrar todos falando, cada um sobre a sua parte.
- **RO-2** Prazo da Entrega 3 do PJBL, com o mesmo sistema das entregas anteriores.
- **RO-3** Documentação em arc42, com C4 Model (contexto, contêineres e componentes), Canvas e prints.
- **RO-4** Recursos de nuvem divididos entre contas dos integrantes (AWS para o gateway; Azure para contêineres, Function, Service Bus e SQL; MongoDB Atlas para identidade).

2.3 Convenções

- Código e mensagens de commit em inglês (restam alguns comentários antigos em português); READMEs e documentação de entrega em português.
- Biome formata e verifica todo o monorepo (`pnpm check`); Vitest roda todos os testes a partir da raiz (`pnpm test`).
- Cada caso de uso é uma pasta `features/<caso-de-uso>` com caso de uso, controller, DTO e teste juntos.
- Contratos: OpenAPI gerado pelo `@nestjs/swagger` a partir de comentários nos DTOs; AsyncAPI 3.0 para os eventos.

3. Contexto e Escopo

3.1 Contexto Negocial

O sistema tem um tipo de usuário, o operador da carteira, que usa o frontend pelo navegador. A equipe de desenvolvimento publica o código no GitHub e as imagens no Docker Hub, de onde o Azure Container Apps baixa cada versão. Os bancos e a fila são serviços gerenciados, mas pertencem ao sistema: aparecem como contêineres no nível 2.

Parceiro	Entrada para o sistema	Saída do sistema
Operador da carteira	Cadastros, aberturas de carteira, depósitos, saques, edições e exclusões	Listas, detalhes, extrato, painel com score e feed de atividade, mensagens de erro de regra de negócio
Equipe de desenvolvimento	Código (GitHub) e imagens versionadas (Docker Hub)	—
Docker Hub	Imagens dos contêineres	Pedidos de download das imagens (docker pull)
GitHub	Código, READMEs e documentação dos eventos	—

3.1 Contexto Negocial – Diagrama de Contexto do Sistema (C4 Nível 1)

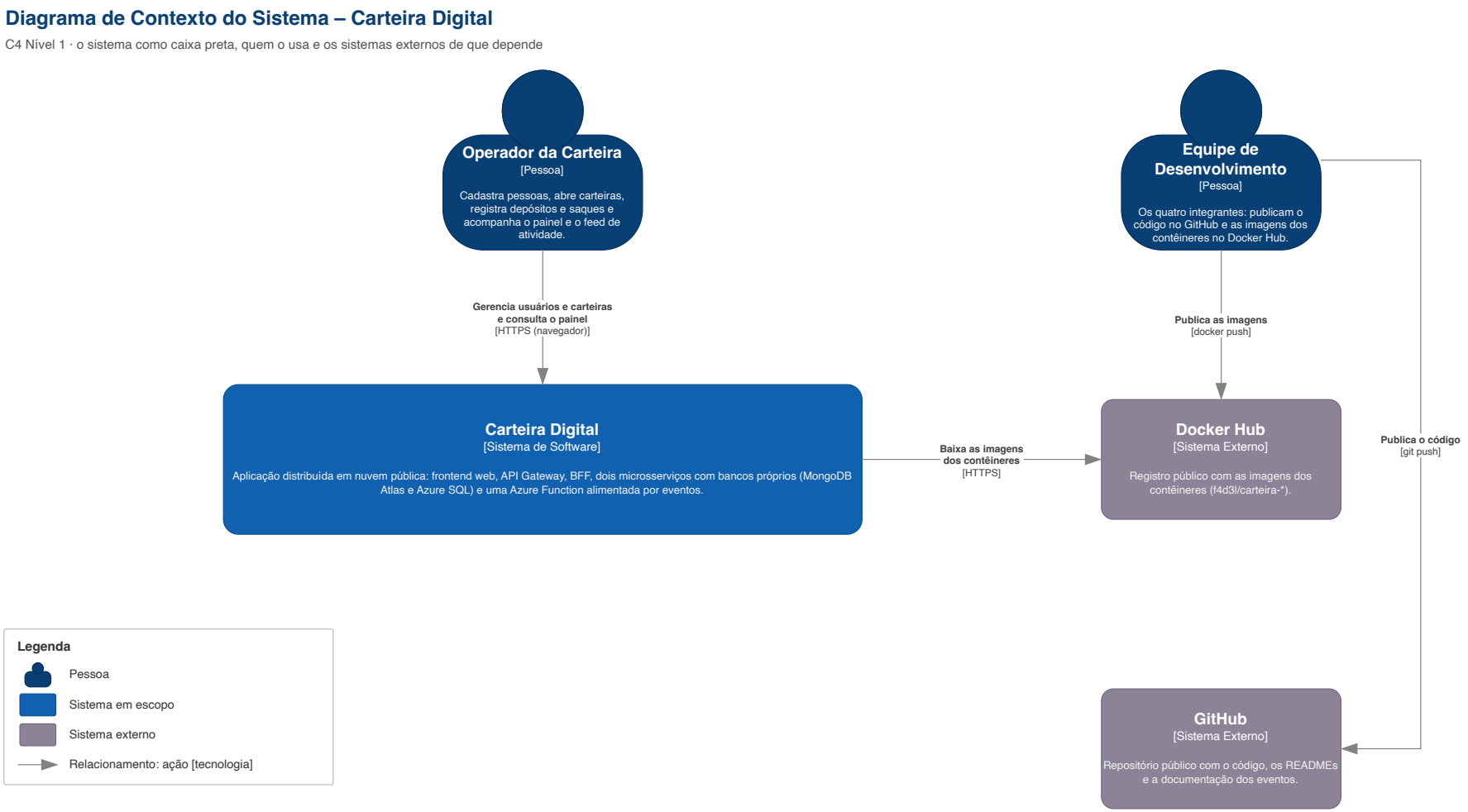


Figura 2 — Nível 1 do C4 Model: contexto do sistema Carteira Digital

3.2 Contexto Técnico

Interface	De → para	Protocolo	Formato
Frontend → API Gateway	Navegador → AWS	HTTPS	JSON (cliente gerado do OpenAPI do BFF)
API Gateway → BFF	AWS → Azure Container Apps	HTTPS, rota `ANY /{proxy+}`	Repassa método, caminho e corpo
BFF → microsserviços e Function	Container Apps → Container Apps e Azure Functions	HTTPS (fetch)	JSON
svc-identidade → svc-carteira	Antes de excluir um usuário	HTTPS	`GET /wallets?ownerId=`
Microsserviços → fila	Container Apps → Azure Service Bus	AMQP (amqps)	Envelope JSON `{id, type, occurredAt, data}`
Fila → Function	Service Bus → walletEvents	Gatilho de Service Bus	Mesmo envelope
Serviços → bancos	svc-identidade → Atlas; svc-carteira → Azure SQL; Function → Table Storage	mongodb+srv/TLS; TDS/TLS 1433; HTTPS	Documentos, tabelas e entidades (seção 8.1)

4. Estratégia de Solução

4.1 Decisões Fundamentais e Estilos Arquiteturais

Estilo ou padrão	Onde aparece	Por quê
Microfrontend (SPA)	`apps/web`	Interface isolada do back-end, que só conhece o contrato do BFF
API Gateway	AWS API Gateway HTTP API	Entrada única: o frontend conhece um só endereço, e o CORS do gateway só aceita o domínio do frontend
Backend for Frontend	`apps/bff`	Uma API sob medida para a interface, que agrega os serviços e esconde a topologia interna
Microsserviços + Database per Service	svc-identidade (MongoDB) e svc-carteira (Azure SQL)	Cada domínio evolui, escala e é implantado sozinho; cada um escolhe o banco que combina com seus dados
Arquitetura orientada a eventos	Fila `wallet-events` no Azure Service Bus	Os serviços avisam o que aconteceu sem conhecer quem consome; a Function processa no seu ritmo
Serverless	fn-notificacoes no Azure Functions	Processamento por evento e cálculo sob demanda, pagos por execução
Clean Architecture + Vertical Slice	Dentro do BFF e de cada microsserviço	Regras no centro, frameworks na borda; cada caso de uso numa fatia própria

4.2 Como os objetivos de qualidade são atingidos

Objetivo	Abordagem
Manutenibilidade	Portas no domínio (classes abstratas) e adaptadores na infraestrutura; casos de uso montados por fábrica no `app.module.ts`, sem anotação de framework; testes de arquitetura com 10 regras por backend
Integridade do saldo	Aritmética em centavos; `UPDATE … WHERE id AND balance = saldo lido` e inserção do lançamento na mesma transação; 0 linhas afetadas vira 409 e nada é gravado
Implantabilidade	Um `Dockerfile.nest` para os três backends (`--build-arg APP=`); backends configurados só por variáveis de ambiente, com segredos no Container App; o frontend recebe a URL do gateway no build
Interoperabilidade	Swagger em `/docs` em cada serviço NestJS; cliente TypeScript, hooks do TanStack Query e schemas zod gerados pelo Kubb a partir do OpenAPI do BFF; AsyncAPI para os eventos

4.3 Mudanças em relação à Entrega 2

Aspecto	Entrega 2 (projeto)	Entrega 3 (implementado)	Motivo
Escopo	Onboarding, PIX, pagamentos, cartões, antifraude e backoffice	Usuários, carteiras, depósito, saque, extrato, feed e score	Recorte fixado pelo enunciado da Entrega 3
Serviços	Seis microsserviços e dois BFFs	Dois microsserviços, um BFF e uma Function	Enunciado pede dois microsserviços com bancos diferentes e um BFF
Linguagem	Java 25 com Spring Boot 4	TypeScript com NestJS 12 (Node.js)	BFF em Node.js pedido pelo enunciado; uma linguagem só para os quatro backends
Bancos	PostgreSQL, um banco por serviço	MongoDB Atlas e Azure SQL	Integração com múltiplos bancos exigida
Gateway	Azure API Management	AWS API Gateway (HTTP API)	Exigência do enunciado
Frontend	App React Native e portal com Module Federation	Uma SPA React	Um único canal de uso no escopo atual
Eventos	Tópicos no Service Bus com outbox	Uma fila no Service Bus, sem outbox	Simplicidade no prazo; risco registrado (R-2)

5. Visão de Blocos de Construção

5.1 Visão Sistêmica Geral de Caixa Branca (C4 Nível 2)

A Figura 3 mostra os contêineres e como conversam. O caminho síncrono vai do frontend ao API Gateway, do gateway ao BFF e do BFF aos dois microsserviços e à Function. O caminho assíncrono vai dos microsserviços à fila `wallet-events` e da fila à Function, que grava o histórico na Table Storage.

Contêiner	Tecnologia	Responsabilidade	Interfaces
Frontend Web	React 19, TanStack Start/Router/Query/Form/Table, Base UI, Kubb	Telas Painel, Usuários, Carteiras e Atividade, com os detalhes de usuário e de carteira	Chama apenas o API Gateway
API Gateway	AWS API Gateway (HTTP API)	Entrada única; rota `ANY /{proxy+}` para o BFF, estágio `$default`; CORS só para o frontend	HTTPS público
BFF	NestJS 12 + Fastify	Proxy do CRUD de usuários e carteiras, `GET /activity` e `GET /aggregated-data`	14 rotas (Swagger em `/docs`)
svc-identidade	NestJS 12 + Mongoose 9	CRUD de usuários; publica `UserCreated`	`/users` e `/users/{id}`
svc-carteira	NestJS 12 + TypeORM 1.1 + mssql	Carteiras, depósito, saque, extrato e encerramento; publica os eventos da carteira	`/wallets` e sub-rotas
fn-notificacoes	Azure Functions v4 (Node.js)	Consome os eventos, grava o histórico, expõe feed e score	`/api/activity`, `/api/insights` e gatilho `walletEvents`
Banco de Identidade	MongoDB Atlas (Free Tier)	Banco `identidade`, coleção `users`	Só o svc-identidade acessa
Banco da Carteira	Azure SQL Database	Tabelas `wallets` e `transactions`	Só o svc-carteira acessa
Fila de Eventos	Azure Service Bus	Fila `wallet-events` com os cinco tipos de evento	Publicadores: os dois serviços; consumidor: a Function
Histórico de Eventos	Azure Table Storage	Tabela `events`	Só a Function acessa

5.1 Diagrama de Contêineres (C4 Nível 2)

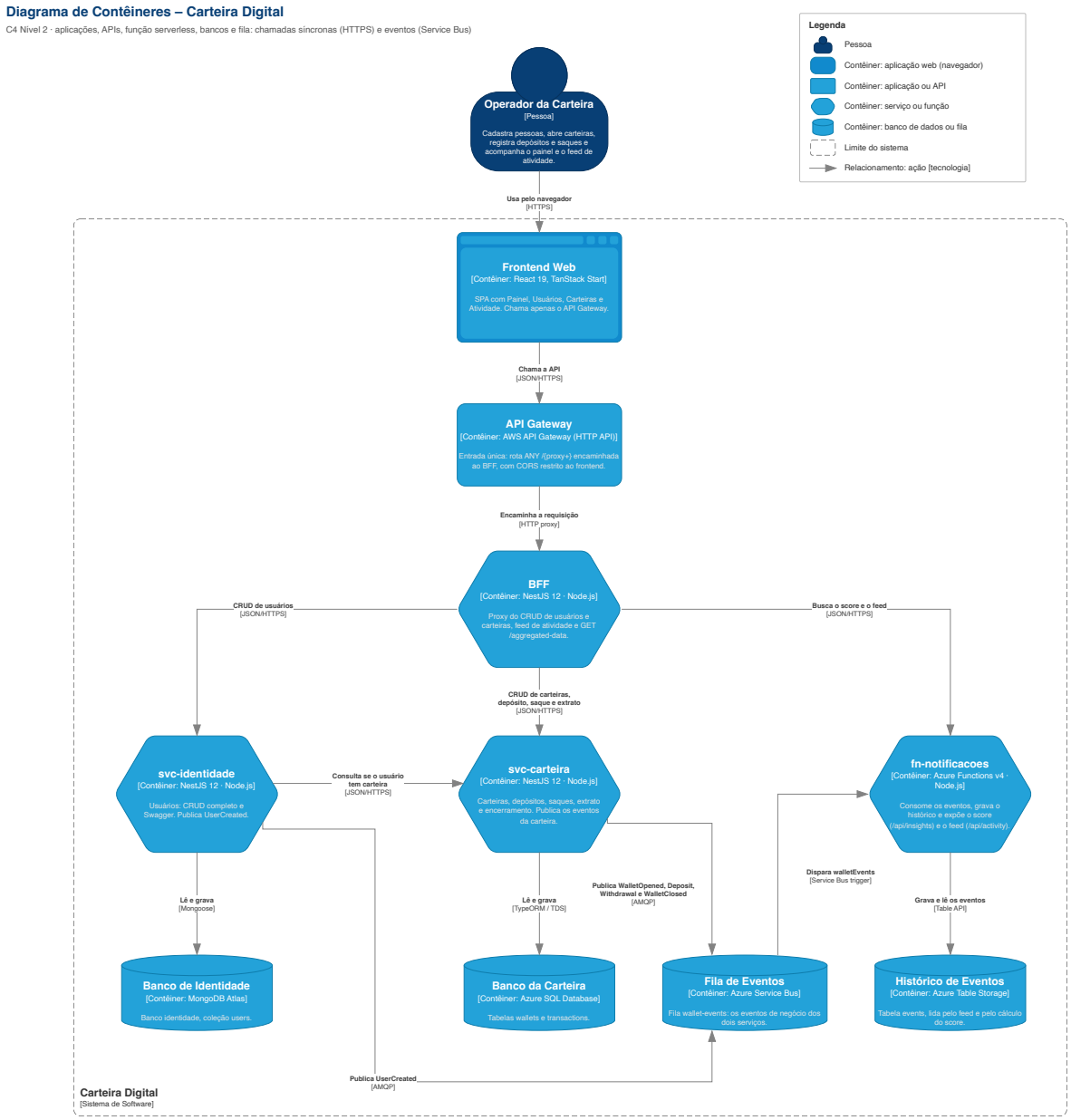


Figura 3 — Nível 2 do C4 Model: contêineres da Carteira Digital

Catálogo de eventos

Todos os eventos usam o envelope `{id, type, occurredAt, data}`, com `messageId = id` e `contentType: application/json`. O contrato está em AsyncAPI 3.0 (`docs/events/asyncapi.yaml`).

Evento	Publicado por	Quando	Campos de `data`
`UserCreated`	svc-identidade	Depois de gravar um usuário novo	`userId, name, email`
`WalletOpened`	svc-carteira	Depois de abrir a carteira	`walletId, ownerId`
`Deposit`	svc-carteira	Depois de gravar o depósito	`walletId, ownerId, amount, balance`
`Withdrawal`	svc-carteira	Depois de gravar o saque	`walletId, ownerId, amount, balance`
`WalletClosed`	svc-carteira	Depois de encerrar a carteira	`walletId, ownerId`

5.2 Componentes do BFF (C4 Nível 3)

O BFF tem 14 fatias finas: cada uma recebe a requisição, chama a porta do serviço certo e devolve o resultado. As portas `IdentityService`, `WalletService` e `NotificationsService` são implementadas por adaptadores HTTP que usam `fetch` e um tradutor comum de respostas (`http-json.ts`): 400, 404, 409 e 422 dos serviços viram erros de domínio, e qualquer outro status ou falha de rede vira `ServiceUnavailableError`. O filtro de erros devolve 400, 404, 409, 422 ou 502 ao frontend, mantendo a mensagem original. Assim, a mensagem "Email … is already in use" que nasce no svc-identidade chega igual ao formulário.

5.2 Diagrama de Componentes do BFF (C4 Nível 3)

Diagrama de Componentes – BFF (Backend for Frontend)

C4 Nível 3 · 14 fatias finas (agrupadas por assunto) sobre Clean Architecture; os serviços são acessados por portas com adaptadores HTTP (fetch)

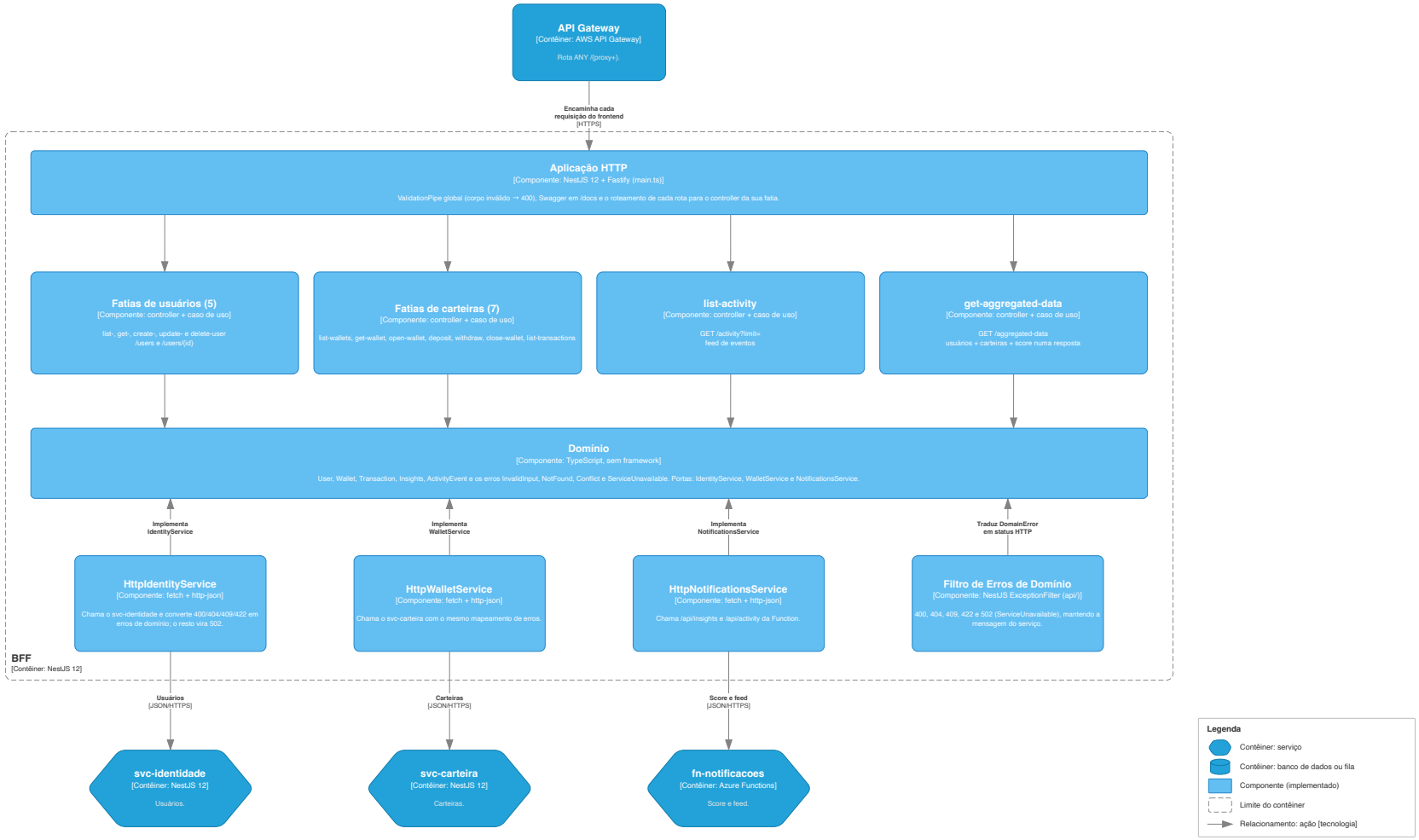


Figura 4 — Nível 3 do C4 Model: componentes do BFF

5.3 Componentes do svc-identidade (C4 Nível 3)

Cinco fatias (`create-user`, `list-users`, `get-user`, `update-user` e `delete-user`) sobre o domínio `User`. As regras ficam nos casos de uso: e-mail único na criação e na edição, e a exclusão só acontece depois de perguntar ao svc-carteira, pela porta `WalletDirectory`, se o usuário tem carteira. Se o svc-carteira não responder, a exclusão falha (500 no svc-identidade, 502 no frontend, via BFF) em vez de apagar um usuário que talvez tenha carteira.

5.3 Diagrama de Componentes do svc-identidade (C4 Nível 3)

Diagrama de Componentes – svc-identidade

C4 Nível 3 · Vertical Slice sobre Clean Architecture: as regras ficam nos casos de uso e no domínio; MongoDB, HTTP e Service Bus são adaptadores

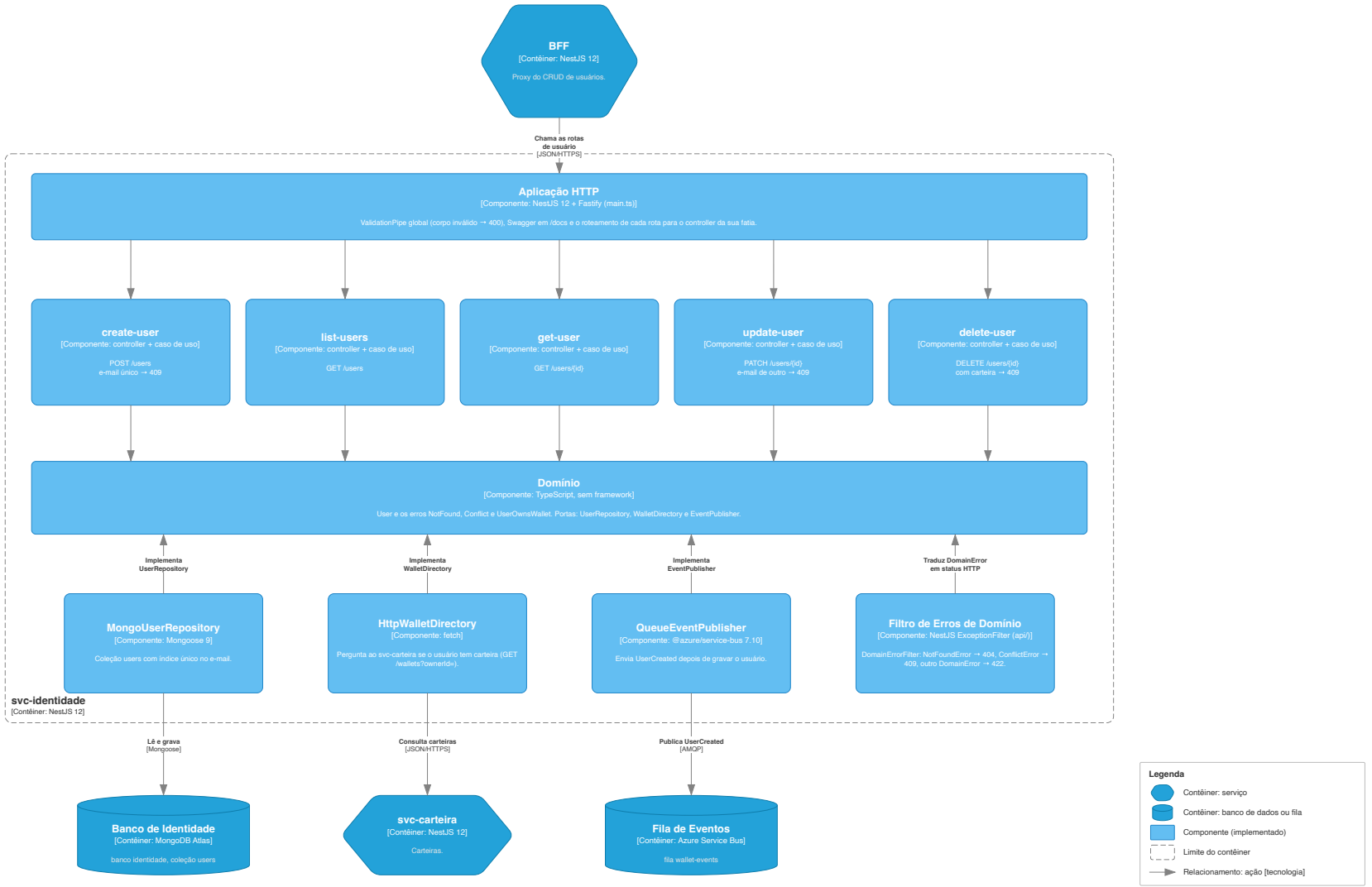


Figura 5 — Nível 3 do C4 Model: componentes do svc-identidade

5.4 Componentes do svc-carteira (C4 Nível 3)

Sete fatias sobre o agregado `Wallet`. O domínio faz as contas em centavos (`credit` e `debit`) e decide se a carteira pode ser encerrada (`assertCanClose`). O `SqlWalletRepository` grava cada movimento numa transação e só atualiza o saldo se ele ainda for o valor lido; o `QueueEventPublisher` publica o evento depois da gravação e, se o broker falhar, só registra no log.

5.4 Diagrama de Componentes do svc-carteira (C4 Nível 3)

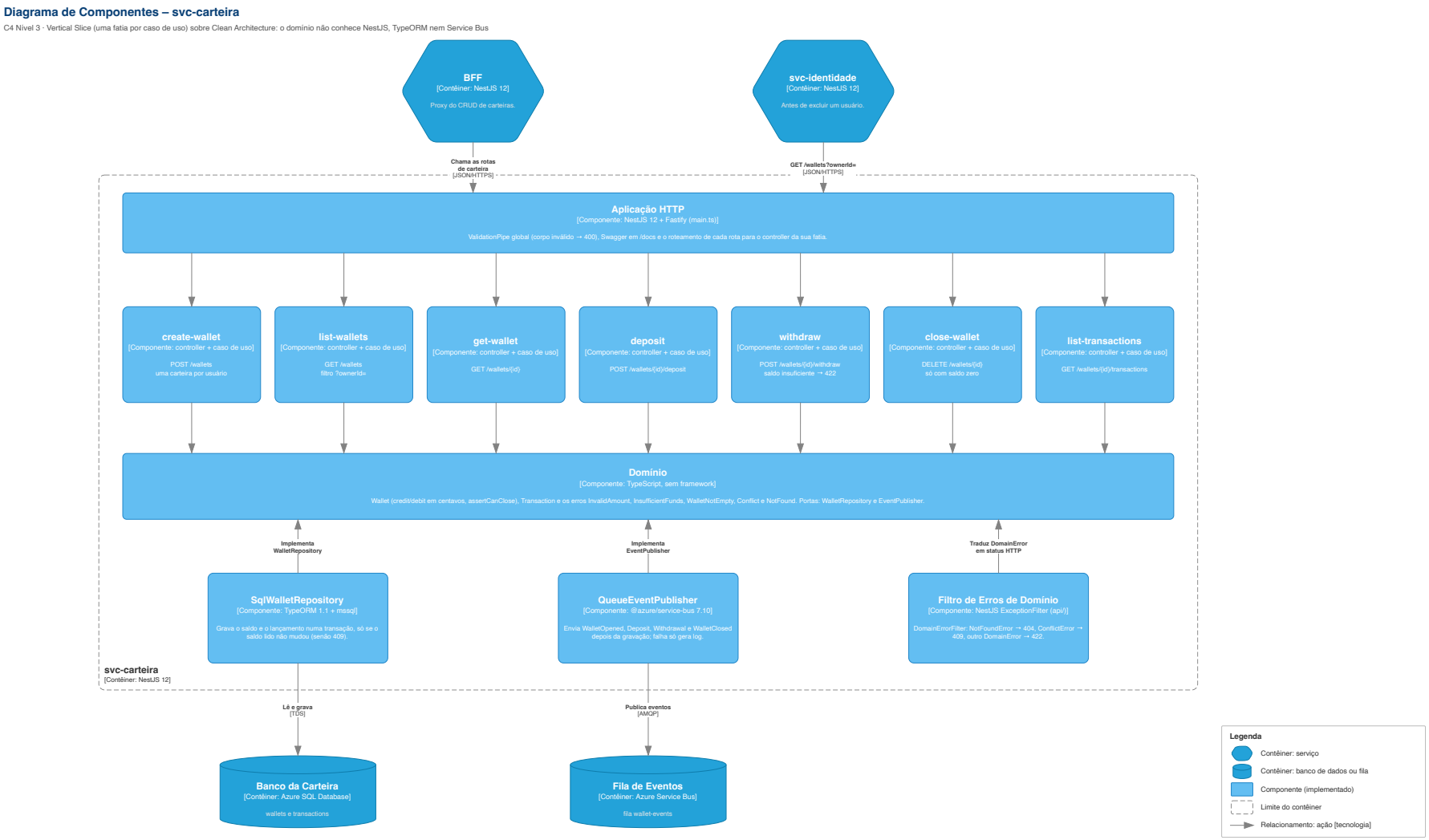


Figura 6 — Nível 3 do C4 Model: componentes do svc-carteira

5.5 Componentes da fn-notificacoes (C4 Nível 3)

A Function tem três gatilhos e lógica pura separada do armazenamento. `walletEvents` valida o envelope (`events.ts`) e grava com `upsert`; `activity` lê os últimos N eventos; `insights` lê os eventos e calcula o score (`score.ts`). Hoje só o `event-store.ts` conhece a Table Storage; o teste de arquitetura próprio garante que a lógica pura (`events.ts` e `score.ts`) não depende de nada externo, que o `event-store.ts` não importa os gatilhos e que um gatilho não importa outro.

5.5 Diagrama de Componentes da fn-notificacoes (C4 Nível 3)

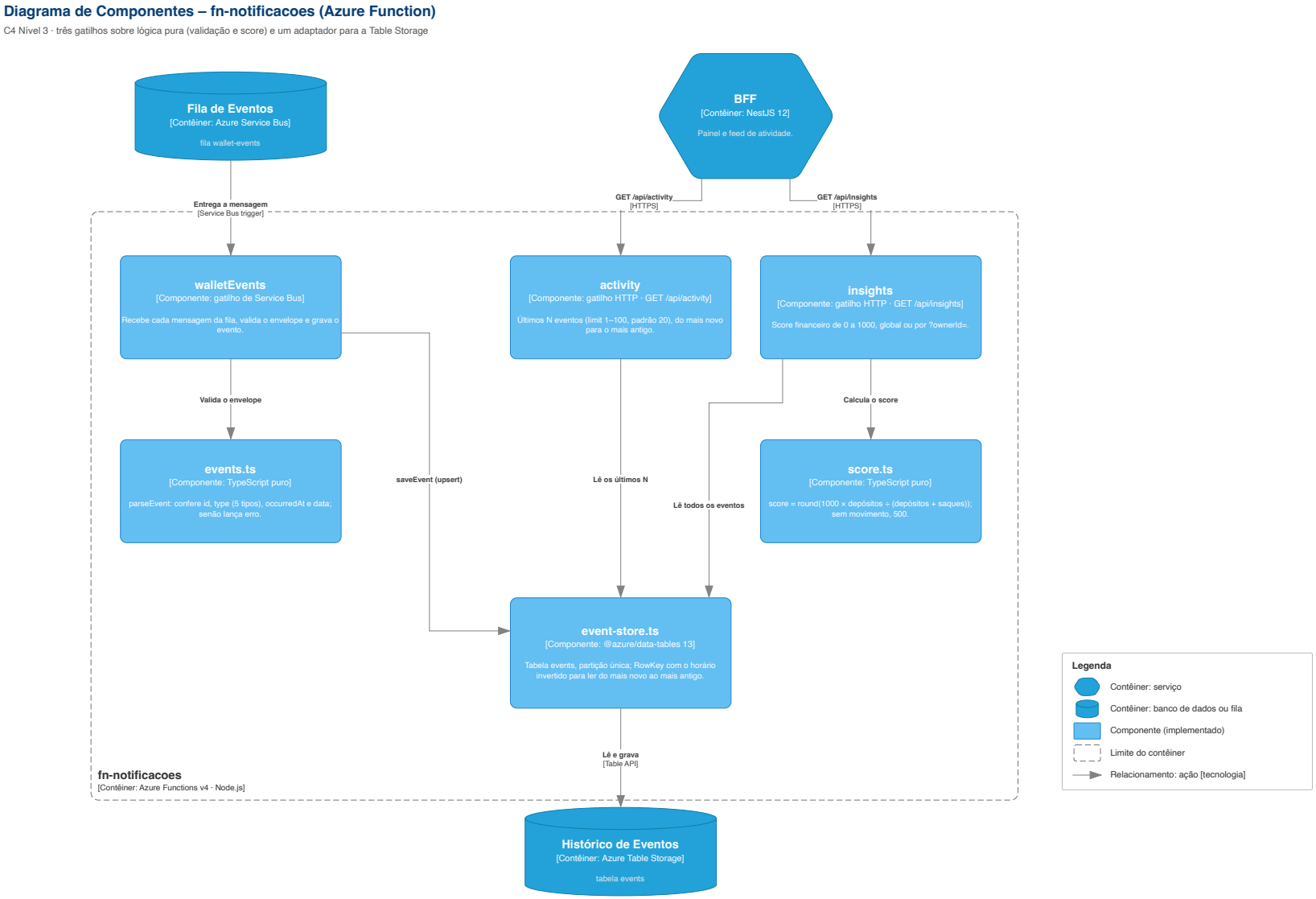


Figura 7 — Nível 3 do C4 Model: componentes da fn-notificacoes

6. Visão de Tempo de Execução

6.1 Cenário: depósito de ponta a ponta

A Figura 8 segue um depósito do clique no frontend até o histórico da Function. Os passos 1 a 19 são síncronos: o BFF valida o valor, o svc-carteira lê a carteira, aplica a regra no domínio e grava saldo e lançamento numa transação. Os dois fragmentos `break` mostram as saídas de erro: valor com mais de 2 casas decimais (422; um valor menor que 0,01 já volta 400 no passo 3) e conflito de concorrência (409). Depois da resposta, o evento `Deposit` já está na fila, e a Function o grava na Table Storage de forma assíncrona. O feed e o score passam a refletir o depósito alguns instantes depois (consistência eventual).

6.1 Diagrama de Sequência UML – depósito

Diagrama de Sequência UML – depósito de ponta a ponta (POST /wallets/{id}/deposit)

Do frontend ao Azure SQL pelo API Gateway e pelo BFF; depois da resposta, o evento Deposit segue pelo Service Bus até a Function

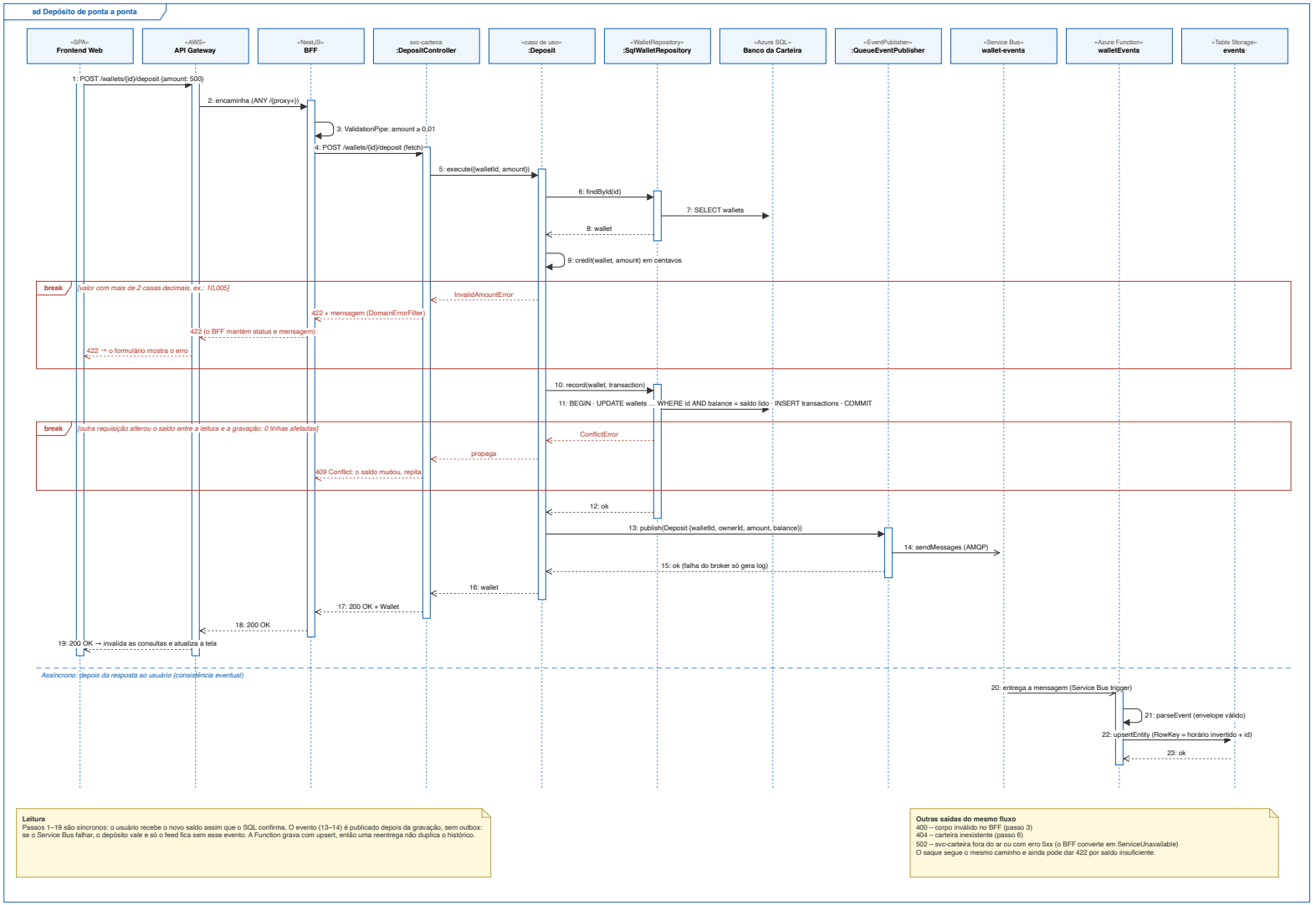


Figura 8 — Sequência UML do depósito de ponta a ponta, com o evento

6.2 Cenário: Painel com GET /aggregated-data

O Painel faz duas chamadas: `GET /aggregated-data` (usuários, carteiras e score) e `GET /activity?limit=5` (atividade recente). No agregado, o BFF dispara três requisições em paralelo (`Promise.all`) para o svc-identidade, o svc-carteira e a Function e responde `{users, wallets, insights}`. Se qualquer fonte falhar, a resposta inteira vira 502 e o Painel marca as três fontes do agregado como indisponíveis, porque não sabe qual delas falhou.

6.2 Diagrama de Sequência UML – aggregated-data

Diagrama de Sequência UML – GET /aggregated-data (Painel)

O BFF consulta os dois microsserviços e a Function em paralelo e devolve um único JSON; se uma fonte falhar, responde 502

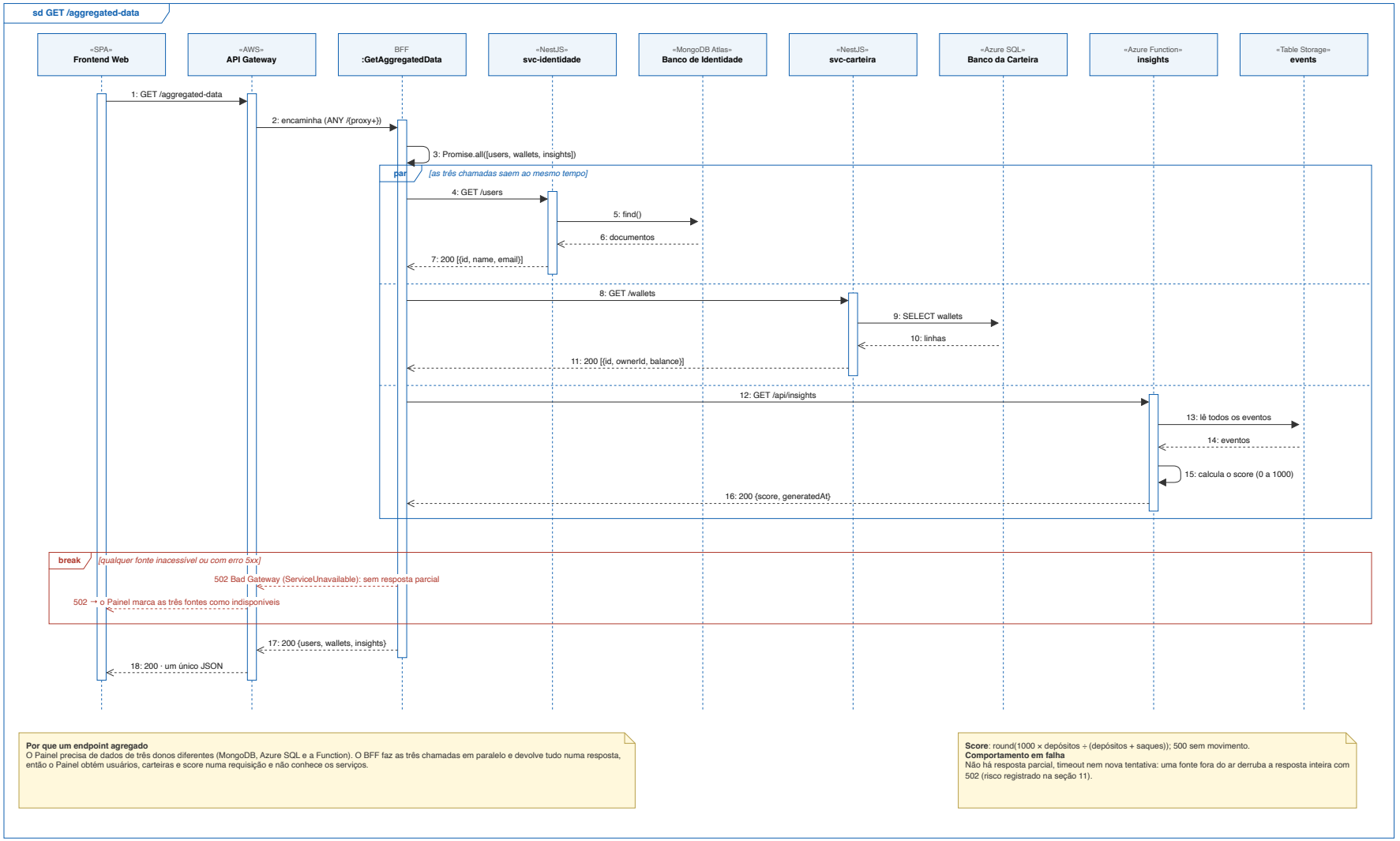


Figura 9 — Sequência UML do GET /aggregated-data

6.3 Cenário: exclusão de usuário que tem carteira

1. O frontend chama `DELETE /users/{id}`; o API Gateway e o BFF repassam ao svc-identidade.
2. O caso de uso `DeleteUser` confirma que o usuário existe (senão 404).
3. Pela porta `WalletDirectory`, o svc-identidade chama `GET /wallets?ownerId={id}` no svc-carteira.
4. A lista não está vazia: o caso de uso lança `UserOwnsWalletError`, o filtro responde 409 e o BFF repassa a mensagem ao frontend.
5. Para excluir, o operador saca o saldo, encerra a carteira (`WalletClosed`) e então exclui o usuário.

7. Visão de Implantação

7.1 Nível de Infraestrutura 1

O API Gateway roda na AWS; frontend, BFF e microsserviços rodam como Container Apps num mesmo ambiente da Azure (Canada Central), com imagens baixadas do Docker Hub; a Function, a fila, a tabela e o SQL são serviços gerenciados da Azure; os usuários ficam no MongoDB Atlas.

7.1 Diagrama de Implantação

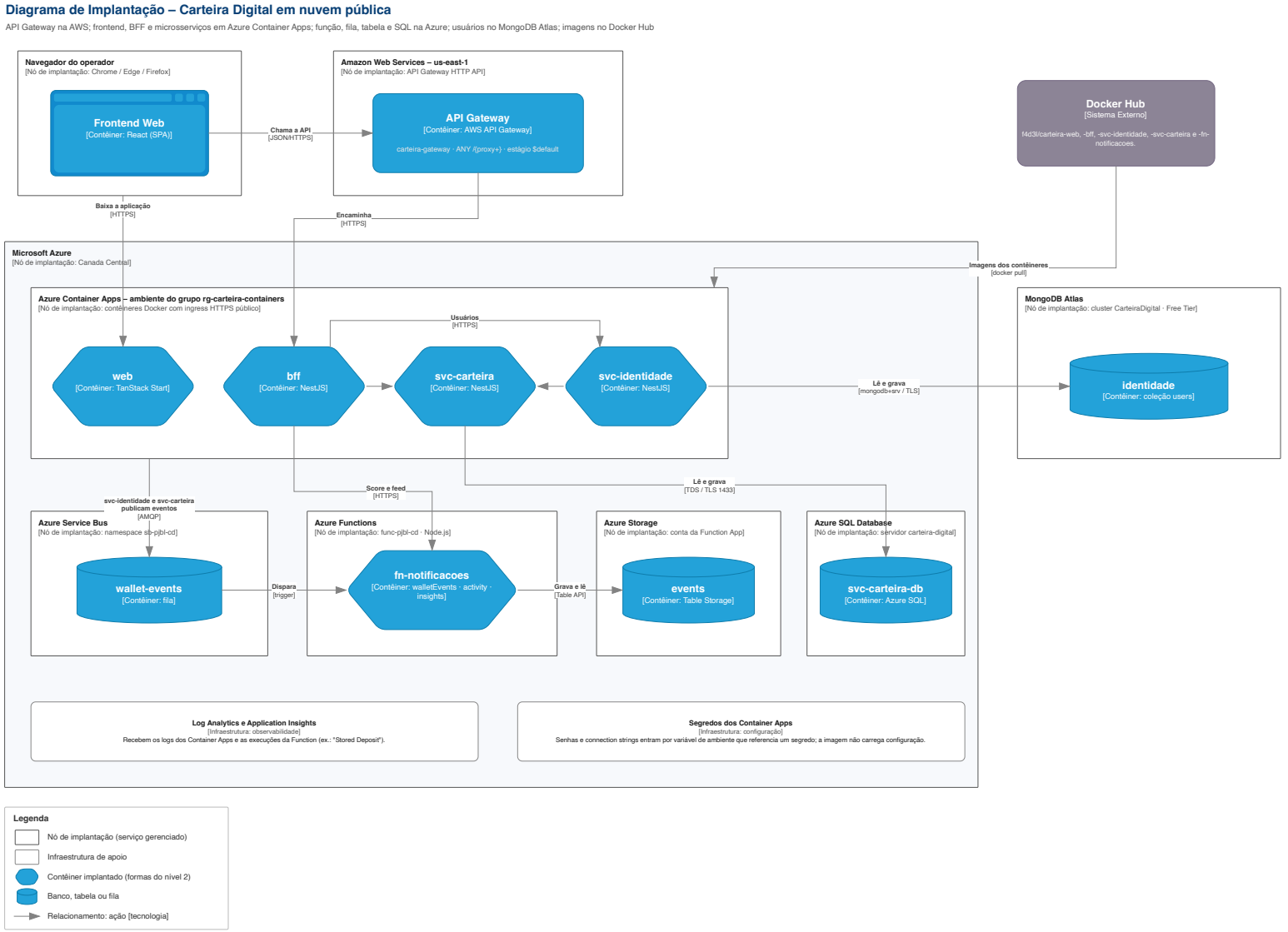


Figura 10 — Implantação da Carteira Digital em nuvem pública

7.2 Recursos e endereços

Elemento	Recurso	Endereço ou identificação
API Gateway	AWS API Gateway HTTP API `carteira-gateway`, us-east-1	`i12uxusdx6.execute-api.us-east-1.amazonaws.com`
web, bff, svc-identidade, svc-carteira	Azure Container Apps, grupo `rg-carteira-containers`, Canada Central	`<app>.wonderfuldune-fe1dc2f4.canadacentral.azurecontainerapps.io`
fn-notificacoes	Azure Functions `func-pjbl-cd`, Node.js	`func-pjbl-cd-fpfbg6c3axgwh5b2.canadacentral-01.azurewebsites.net`
Fila de eventos	Azure Service Bus `sb-pjbl-cd`, fila `wallet-events`	AMQP
Banco da Carteira	Azure SQL Database, servidor `carteira-digital`	Banco `svc-carteira-db`
Histórico de eventos	Azure Table Storage da Function App	Tabela `events`
Banco de Identidade	MongoDB Atlas, cluster `CarteiraDigital` (Free Tier)	Banco `identidade`
Imagens	Docker Hub, conta `f4d3l`	`carteira-web`, `-bff`, `-svc-identidade`, `-svc-carteira`, `-fn-notificacoes`

7.3 Publicação de uma nova versão

```
docker build -f Dockerfile.nest --build-arg APP=svc-carteira -t f4d3l/carteira-svc-carteira:v2 .
docker push f4d3l/carteira-svc-carteira:v2
# no Azure Container Apps: nova revisão apontando para a tag publicada
```

O frontend recebe a URL do gateway no build (`--build-arg VITE_BFF_URL=…`), porque o Vite embute a variável no pacote. Os backends recebem tudo em tempo de execução: URLs dos serviços, connection strings e senhas, estas como referência a segredos do Container App. Antes de gravar a demonstração, a equipe deixa uma réplica mínima em cada Container App para evitar partida a frio (R-4).

8. Conceitos Transversais

8.1 Modelo de Dados

Cada serviço tem seu próprio banco e nenhum outro serviço o acessa. Entre bancos não há chave estrangeira: `wallets.ownerId` guarda o `_id` do usuário no MongoDB, e a coerência é mantida pelos serviços (o svc-identidade recusa excluir quem tem carteira). No Azure SQL, o schema é criado pelo TypeORM; o índice único em `ownerId` reforça a regra de uma carteira por usuário.

8.1 Diagrama de Entidades e Relacionamentos

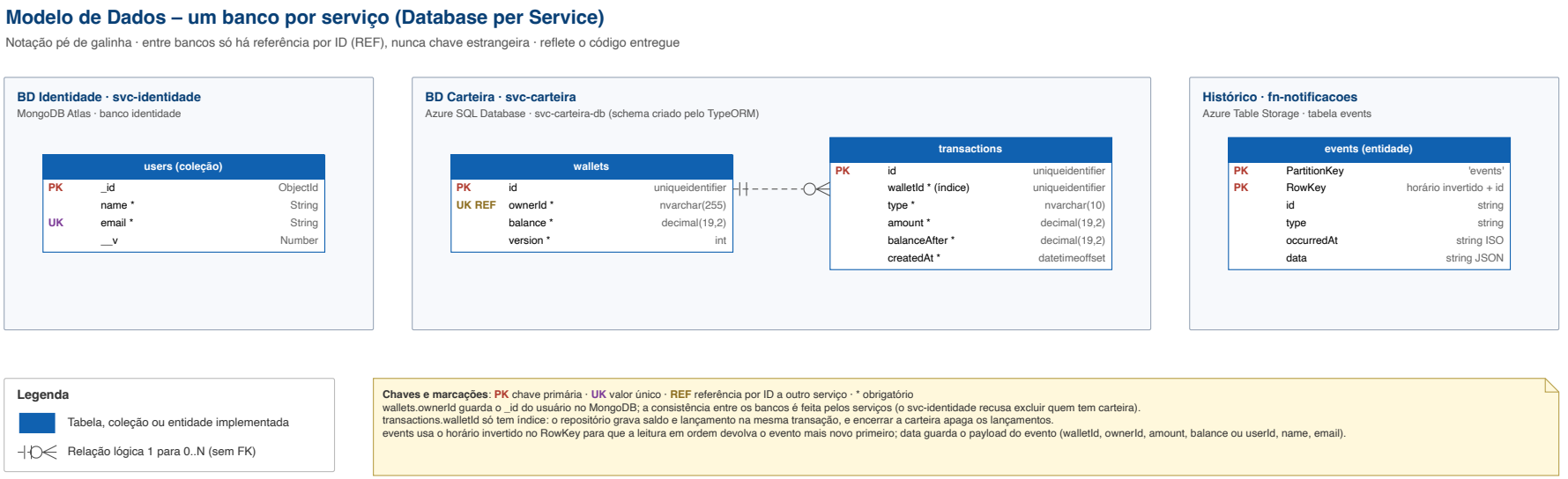


Figura 11 — Modelo de dados da Carteira Digital, um banco por serviço

8.2 Clean Architecture e Vertical Slice

As quatro camadas pedidas no enunciado correspondem a estas partes do código:

Camada	No código
Domain	`domain/` : entidades, portas (classes abstratas) e erros de domínio
Application	`features/<caso-de-uso>/*.use-case.ts` : um caso de uso por fatia, dependendo só das portas
Infrastructure	`infrastructure/` : adaptadores de banco, fila e HTTP que implementam as portas
API	`features/<caso-de-uso>/*.controller.ts` e `*.dto.ts` , mais `api/domain-error.filter.ts`

O BFF e os dois microsserviços seguem a mesma estrutura:

```
src/
  domain/          entidades, portas (classes abstratas) e errors.ts — não importa nada fora de domain/
  features/
    deposit/       deposit.use-case.ts · deposit.controller.ts · deposit.dto.ts · deposit.use-case.spec.ts
    withdraw/      …uma pasta por caso de uso
  infrastructure/  adaptadores: SqlWalletRepository, QueueEventPublisher, MongoUserRepository, HttpWalletDirectory…
  api/             domain-error.filter.ts (DomainError → HTTP)
  testing/         repositórios em memória para os testes
  app.module.ts    composição: { provide: Porta, useClass: Adaptador } e casos de uso por fábrica
  main.ts          Fastify, ValidationPipe global e Swagger em /docs
```

O caso de uso é uma classe comum com um método `execute` , que depende só das portas do domínio; o controller injeta o caso de uso, nunca o repositório; uma fatia nunca importa outra. As regras verificadas pelo `architecture.spec.ts` em cada backend são:

1. Todo arquivo pertence a `domain` , `features` , `infrastructure` , `api` ou `testing` (ou é `main.ts` / `app.module.ts`).
2. `domain/` só importa `domain/` .
3. Um caso de uso só importa o domínio ou a própria fatia.
4. Controllers só importam NestJS, a própria fatia e entidades; nunca infraestrutura nem portas.
5. DTOs só importam class-validator, class-transformer, `@nestjs/swagger` ou entidades.
6. Uma fatia nunca importa outra.
7. `infrastructure/` nunca importa `features/` nem `api/` .
8. `api/` só importa NestJS e o domínio.
9. Os fakes de `testing/` não importam frameworks, infraestrutura, features nem api.
10. Código de produção nunca importa `testing/` .

8.3 Tratamento de erros

Erro de domínio	Exemplo	Microsserviço	BFF
Validação do DTO	Corpo sem `email` ; no BFF, depósito menor que 0,01	400 (no svc-carteira, depósito de 0 dá 422)	400
`NotFoundError`	Usuário ou carteira inexistente	404	404
`ConflictError` e subclasses	E-mail em uso; segunda carteira; encerrar com saldo; excluir usuário com carteira; saldo alterado em paralelo	409	409
Outro `DomainError`	Valor com 3 casas; saldo insuficiente	422	422
`ServiceUnavailableError`	Serviço fora do ar ou com 5xx	—	502

Nenhuma camada fora do controller lança exceção HTTP: os casos de uso lançam erros de domínio e o `DomainErrorFilter` de cada aplicação faz a tradução.

8.4 Comunicação por eventos

- **Transporte:** uma fila do Azure Service Bus (`wallet-events`) para os cinco tipos. Os publicadores usam `SERVICEBUS_CONNECTION_STRING` ; a Function usa a configuração `ServiceBusConnection` .

- **Momento:** o evento é publicado depois da gravação no banco. No svc-carteira, falha do broker só gera log; no svc-identidade, a falha sobe como erro 500, embora o usuário já esteja gravado (R-7).
- **Consumo:** o gatilho `walletEvents` valida o envelope e grava com `upsert`, então uma reentrega da mesma mensagem não duplica o histórico. Uma mensagem inválida faz a função lançar erro: o Service Bus a reentrega até o limite de tentativas e depois a move para a fila de mensagens mortas (dead-letter).
- **Ordenação de leitura:** o `RowKey` começa pelo horário invertido, então ler a tabela na ordem natural já devolve o evento mais novo primeiro.
- **Contrato:** AsyncAPI 3.0 em `docs/events/asyncapi.yaml`, citado como "Swagger de Eventos" na lista de links.

8.5 Contratos e cliente gerado

O OpenAPI do BFF é gerado pelo plugin do `@nestjs/swagger` a partir dos comentários dos DTOs e fica versionado em `apps/bff/openapi.json`. O Kubb lê esse arquivo e gera, em `apps/web/src/gen`, os tipos, o cliente `fetch` com `baseURL = VITE_BFF_URL` e os hooks do TanStack Query. O cliente gerado fica versionado: depois de regenerá-lo, uma rota removida ou renomeada quebra o build do frontend e um campo alterado aparece como erro do TypeScript, antes de chegar à tela.

8.6 Configuração e segredos

Contêiner	Variáveis
web	`VITE_BFF_URL` (no build)
bff	`SVC_IDENTIDADE_URL`, `SVC_CARTEIRA_URL`, `FN_NOTIFICACOES_URL`
svc-identidade	`MONGO_URL`, `SVC_CARTEIRA_URL`, `SERVICEBUS_CONNECTION_STRING`
svc-carteira	`DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `DB_TRUST_CERT`, `SERVICEBUS_CONNECTION_STRING`
fn-notificacoes	`ServiceBusConnection`, `AZURE_STORAGE_CONNECTION_STRING`, `AzureWebJobsStorage`

Nos Container Apps, senhas e connection strings ficam como segredos referenciados pelas variáveis; na Function, como configurações de aplicativo. Nenhuma imagem dos backends carrega configuração, por isso a mesma imagem do BFF e dos microsserviços roda no `docker compose` local e na nuvem. A exceção é o frontend, que embute `VITE_BFF_URL` no build (seção 7.3).

8.7 Estratégia de testes

Projeto	Arquivos de teste	Testes	O que cobrem
bff	16	47	14 casos de uso, tradutor de respostas HTTP e arquitetura
svc-identidade	6	22	5 casos de uso (incluindo e-mail repetido e exclusão com carteira) e arquitetura
svc-carteira	9	38	7 casos de uso (depósito e saque com 7 testes cada), validação do formato de GUID do SQL Server e arquitetura
fn-notificacoes	3	10	Validação do envelope, cálculo do score e arquitetura
Total	**34**	**117**	`pnpm test` na raiz: 117 aprovados, 0 falhas

Os testes unitários instanciam o caso de uso com repositórios em memória (`src/testing/`), sem banco nem injeção de dependência do NestJS, e cobrem o caminho feliz e cada erro de domínio. O frontend não tem testes automatizados.

8.8 Segurança

Controle	Como está
Entrada única com HTTPS	O frontend só conhece o API Gateway; todo o tráfego é HTTPS.

CORS no API Gateway	Só a origem `https://web.wonderfuldune-fe1dc2f4.canadacentral.azurecontainerapps.io` recebe permissão, com os métodos GET, POST, PATCH, DELETE e OPTIONS e o cabeçalho `content-type` (cache do preflight de 300 s). Com CORS configurado, o gateway ignora os cabeçalhos de CORS devolvidos pelo BFF. Uma página de outro domínio não recebe a permissão e o navegador bloqueia a chamada (conferido com requisições de preflight).
Segredos	Senhas e connection strings ficam como segredos dos Container Apps e nas configurações de aplicativo da Function App; nada no código nem nas imagens.
Validação de entrada	ValidationPipe com `whitelist` nos três backends NestJS: campos desconhecidos são descartados e corpos inválidos recebem 400.
Isolamento dos dados	Cada banco só é acessado pelo seu serviço (Database per Service), com credenciais próprias.

O CORS protege as chamadas feitas por navegadores. Ainda não há autenticação de usuários nem limite de requisições no gateway, e o BFF e os dois microsserviços também respondem pelas URLs públicas dos seus Container Apps; ali o CORS do gateway não vale, e o BFF aceita qualquer origem. Essas lacunas estão no risco R-1, com os próximos passos.

8.9 Observabilidade

Os Container Apps enviam logs ao Log Analytics; as execuções da Function aparecem no Application Insights (por exemplo, "Stored Deposit"), e o Service Bus mostra métricas de mensagens recebidas e concluídas. No Painel, os indicadores de origem mostram se o `GET /aggregated-data` respondeu (Identidade, Carteira e Insights, juntos) e se o feed `GET /activity` respondeu (Eventos).

9. Decisões Arquiteturais

ADR-01: Monorepo com cinco pacotes implantados separadamente

Contexto	Cinco partes (frontend, BFF, dois microsserviços e Function) evoluídas por quatro pessoas, com contratos compartilhados.
Decisão	Um repositório pnpm com `apps/*` e `services/*`; cada pacote vira uma imagem e um recurso de nuvem próprios.
Consequências	Um único `pnpm test`, Biome e TypeScript comuns e mudança de contrato visível num commit. O enunciado fala em repositórios públicos por projeto: o README e a lista de links apontam a pasta de cada um.

ADR-02: NestJS + Fastify com o mesmo layout nos três backends

Contexto	O BFF precisa ser Node.js; os microsserviços podiam usar outra stack.
Decisão	NestJS 12 sobre Fastify no BFF e nos dois serviços, com as mesmas pastas, um filtro de erros equivalente (o do BFF acrescenta 400 e 502) e o mesmo teste de arquitetura.
Consequências	Qualquer integrante entende qualquer serviço; um único `Dockerfile.nest`. Em troca, os serviços não exploram outra linguagem.

ADR-03: Database per Service com MongoDB e Azure SQL

Contexto	O enunciado pede integração com MongoDB Atlas e Azure SQL.
Decisão	Usuários (documentos simples, e-mail único) no MongoDB; carteiras e lançamentos (saldo e transações) no Azure SQL. Referência entre eles só por ID.
Consequências	Transações ACID onde o dinheiro está. Sem chave estrangeira entre bancos, a regra "não excluir quem tem carteira" virou uma chamada síncrona do svc-identidade ao svc-carteira.

ADR-04: Eventos numa fila do Azure Service Bus consumida pela Function

Contexto	Feed e score dependem do que acontece nos dois serviços, sem acoplá-los à Function.
Decisão	Uma fila `wallet-events` para os cinco tipos, com envelope comum; a Function consome por gatilho e grava na Table Storage. O transporte começou em Storage Queue e foi trocado por Service Bus.
Consequências	Serviços desacoplados do consumidor e mensagens retidas se a Function estiver fora. Sem outbox, um evento pode se perder se o broker falhar depois da gravação (R-2).

ADR-05: Contrato OpenAPI do BFF e cliente gerado no frontend

Contexto	O frontend só pode falar com o BFF, e erros de contrato aparecem tarde quando escritos à mão.
Decisão	OpenAPI gerado dos DTOs e versionado; Kubb gera tipos, cliente e hooks.
Consequências	Depois de regenerar o cliente, mudanças de rota quebram o build do frontend, e não a tela. É preciso regenerar em ordem (`pnpm --filter bff openapi`, depois `pnpm --filter web generate`).

ADR-06: Azure Container Apps para os contêineres e AWS API Gateway na frente

Contexto	Enunciado exige AWS API Gateway, imagens no Docker Hub e URLs de nuvem; orçamento acadêmico.
Decisão	HTTP API da AWS com rota `ANY /{proxy+}` para o BFF; web, BFF e serviços em Container Apps puxando do Docker Hub; Function em Azure Functions.
Consequências	Gateway simples e barato, que concentra o CORS (restrito ao domínio do frontend). Tem limite de 30 s por integração: a partida a frio de um contêiner pode virar 503 (R-4). Os Container Apps têm ingress público, então o gateway pode ser contornado (R-1).

ADR-07: Saldo em centavos com controle otimista de concorrência

Contexto	Depósitos e saques simultâneos na mesma carteira não podem perder atualização nem deixar saldo negativo.
Decisão	Contas em centavos inteiros no domínio; o repositório atualiza o saldo só se ele for igual ao lido e insere o lançamento na mesma transação.
Consequências	Sem bloqueio pessimista na leitura e sem erro de arredondamento; a requisição que perde a disputa recebe 409 e pode ser repetida.

10. Requisitos de Qualidade

10.1 Árvore de Qualidade

Qualidade	Refinamento	Cenários
Manutenibilidade	Isolamento de camadas	C-1
	Testabilidade	C-2
Integridade do saldo	Regras de negócio	C-3
	Concorrência	C-4
Implantabilidade	Troca de versão independente	C-5
Disponibilidade e resiliência	Falha do consumidor de eventos	C-6
	Falha de uma fonte do Painel	C-7
Segurança	Origem das chamadas no navegador	C-8
Interoperabilidade	Contrato do BFF com o frontend	C-9

Disponibilidade e segurança aparecem como qualidades secundárias: não estão entre os objetivos da seção 1.2, mas ganharam cenários porque o sistema é distribuído e público.

10.2 Cenários de Qualidade

#	Estímulo	Resposta esperada	Situação
C-1	Um integrante importa o repositório SQL direto num controller	`architecture.spec.ts` falha no `pnpm test`	Atendido
C-2	Nova regra num caso de uso	Testada com repositório em memória, sem banco, em milissegundos	Atendido (117 testes)
C-3	Saque de R$ 5.000 numa carteira com R$ 750	422 "Insufficient funds: balance 750, requested 5000"; saldo e extrato inalterados	Atendido
C-4	Dois depósitos simultâneos na mesma carteira	Um grava; o outro recebe 409 e nenhum valor se perde	Atendido no código (UPDATE condicional); sem teste automatizado de concorrência
C-5	Nova versão do svc-carteira	Nova imagem no Docker Hub e nova revisão do Container App, sem tocar nos outros contêineres	Atendido
C-6	Function fora do ar durante depósitos	Mensagens ficam na fila e são processadas quando a Function volta	Atendido pelo Service Bus
C-7	svc-carteira fora do ar ao abrir o Painel	Ideal: Painel parcial. Hoje: 502 e as fontes aparecem offline	Parcial (R-3)
C-8	Uma página de outro domínio tenta chamar a API pelo navegador	O gateway não devolve a permissão de CORS e o navegador bloqueia a chamada	Atendido (clientes fora do navegador: R-1)
C-9	Uma rota do BFF é renomeada	Depois de regenerar o cliente (Kubb), o build do frontend acusa a diferença antes de a tela quebrar	Atendido (exige regenerar o cliente)

11. Riscos e Débitos Técnicos

#	Risco ou débito	Impacto	Mitigação ou próximo passo
R-1	Sem autenticação nem limite de requisições; o BFF e os dois microsserviços também respondem pelas URLs públicas dos Container Apps, e ali o BFF aceita CORS de qualquer origem	Um cliente fora do navegador, ou que chame os contêineres direto, consegue alterar dados	Feito: CORS do gateway restrito ao frontend. Próximos: autorizador JWT e limite de requisições no estágio do gateway; ingress interno nos microsserviços; BFF aceitando só o gateway, com CORS restrito também nele
R-2	Evento publicado depois da gravação, sem outbox	Se o broker falhar, o feed e o score perdem aquele evento	Tabela de outbox gravada na mesma transação e publicada por um processo separado
R-3	`/aggregated-data` responde 502 inteiro se uma fonte falhar; sem timeout nem nova tentativa	Painel vazio quando um serviço oscila	`Promise.allSettled` com resposta parcial e timeout por fonte
R-4	Partida a frio nas camadas gratuitas; o gateway corta em 30 s	503 na primeira chamada depois de um período ocioso	Réplica mínima nos Container Apps durante a demonstração
R-5	Abrir carteira não confere se o `ownerId` existe no svc-identidade	Carteira órfã se a chamada vier de fora do frontend	Consulta ao svc-identidade (ou ao evento `UserCreated`) antes de abrir
R-6	Excluir usuário depende do svc-carteira estar no ar	Exclusão falha durante a indisponibilidade (500 no svc-identidade, 502 no frontend)	Manter: é a escolha segura; mensagem mais clara no futuro
R-7	No svc-identidade, falha ao publicar `UserCreated` devolve 500 (502 no frontend) com o usuário já gravado	O operador pode tentar de novo e receber 409	Mesmo tratamento do svc-carteira (log) ou outbox
R-8	Encerrar carteira apaga o extrato, mas os eventos continuam na Table Storage e no score; o score é global	Score inclui carteiras encerradas	Filtrar por carteiras ativas ou registrar o encerramento no cálculo
R-9	Infraestrutura criada pelos portais, sem IaC; frontend sem testes automatizados	Recriar o ambiente depende de passos manuais	Bicep/Terraform e testes de componente no frontend

12. Glossário

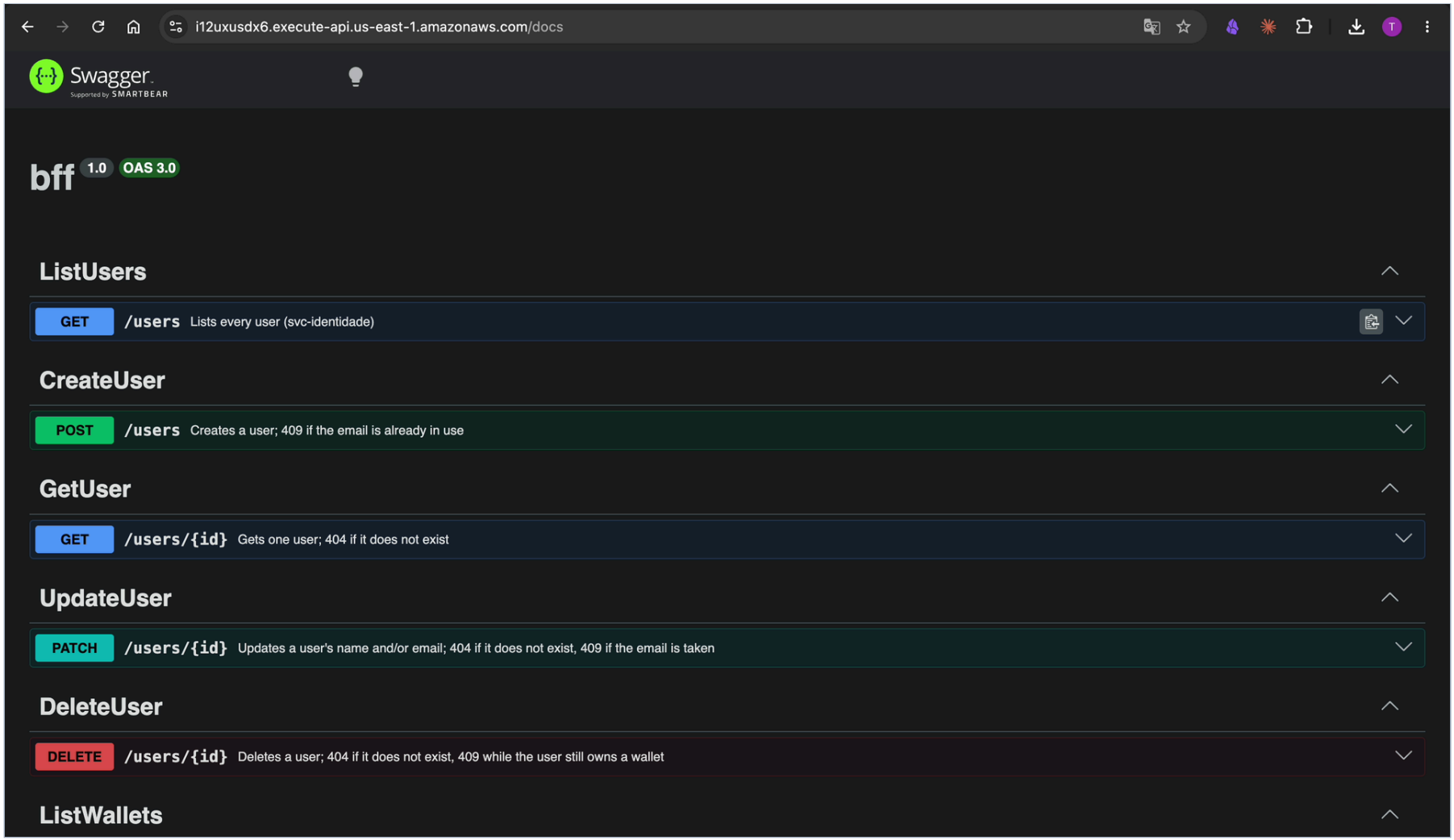
Termo	Definição
API Gateway	Ponto de entrada único das requisições externas; aqui, o HTTP API da AWS que repassa tudo ao BFF.
BFF (Backend for Frontend)	API feita para uma interface específica, que agrega e adapta os serviços internos.
Microsserviço	Serviço pequeno, com domínio e banco próprios, implantado de forma independente.
Database per Service	Cada serviço é dono do seu banco; os outros só acessam os dados pela API dele.
Clean Architecture	Organização em camadas em que as dependências apontam para o domínio, que não conhece frameworks.
Vertical Slice	Organização do código por caso de uso: cada fatia reúne controller, caso de uso, DTO e teste.
Porta e adaptador	Interface definida no domínio (porta) e sua implementação concreta na infraestrutura (adaptador).
Evento	Registro imutável de algo que aconteceu (ex.: `Deposit`), publicado na fila `wallet-events`.
Consistência eventual	O feed e o score refletem uma operação alguns instantes depois dela, quando o evento é processado.
Outbox	Padrão que grava o evento na mesma transação dos dados e o publica depois, para não perdê-lo.
Azure Function	Código serverless executado por gatilho (HTTP ou mensagem) e cobrado por execução.
Container App	Serviço da Azure que roda contêineres com HTTPS, escala e revisões.
Partida a frio	Atraso da primeira requisição quando o contêiner ou a função estava parado.
Score financeiro	Índice de 0 a 1000 calculado pela Function: `round(1000 × depósitos ÷ (depósitos + saques))`.
OpenAPI / Swagger	Contrato das APIs HTTP e a interface web que o exibe em `/docs`.
AsyncAPI	Contrato equivalente ao OpenAPI para mensagens e eventos.
Kubb	Gerador que cria tipos, cliente e hooks a partir do OpenAPI.

Apêndice A. Prints da solução em nuvem

Prints tirados em novembro de 2026, das URLs públicas listadas no início do documento e dos consoles de nuvem: a maioria vem dos vídeos da entrega; os do BFF foram feitos direto no navegador. Os quatro primeiros grupos atendem aos prints exigidos: Swagger do BFF (A.1 e A.2), Swagger dos microsserviços (A.3 e A.4), frontend funcionando (A.5 a A.7) e API Gateway (A.8). Os demais mostram os dados nos bancos e a publicação das imagens.

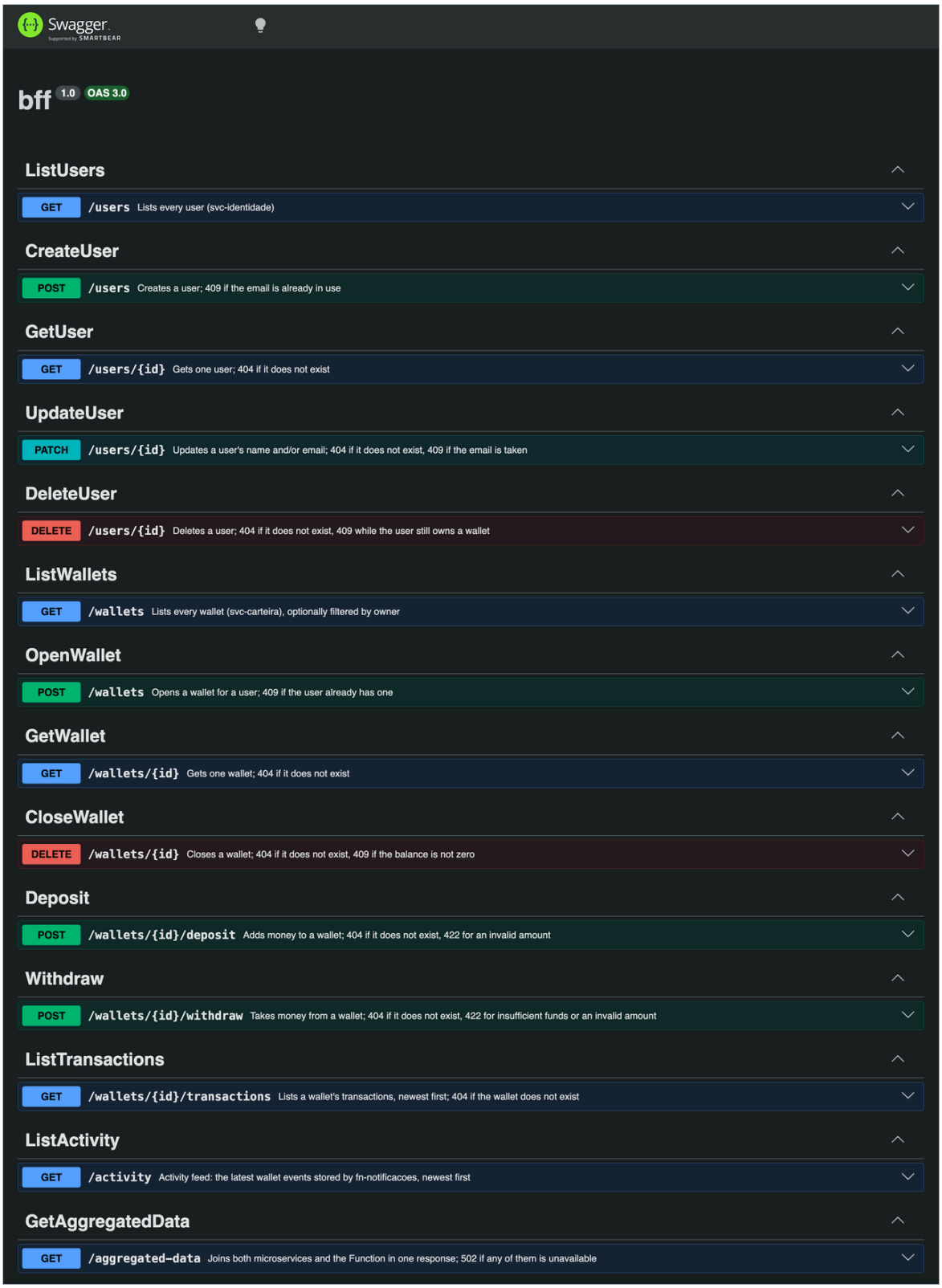
Print	O que mostra
A.1 e A.2	Swagger do BFF acessado pela URL do API Gateway: rotas de usuários, carteiras, atividade e `GET /aggregated-data`
A.3 e A.4	Swagger do svc-identidade e do svc-carteira, com o CRUD completo
A.5 a A.7	Frontend: Painel com dados agregados, detalhe de usuário e detalhe de carteira com extrato
A.8	AWS API Gateway: rota `ANY /{proxy+}` do `carteira-gateway`, com a integração anexada à rota
A.9 e A.10	MongoDB Atlas (coleção `users`) e Azure SQL (tabela `transactions`)
A.11 e A.12	Docker Hub com as cinco imagens e o grupo de recursos dos Container Apps na Azure

Swagger — BFF

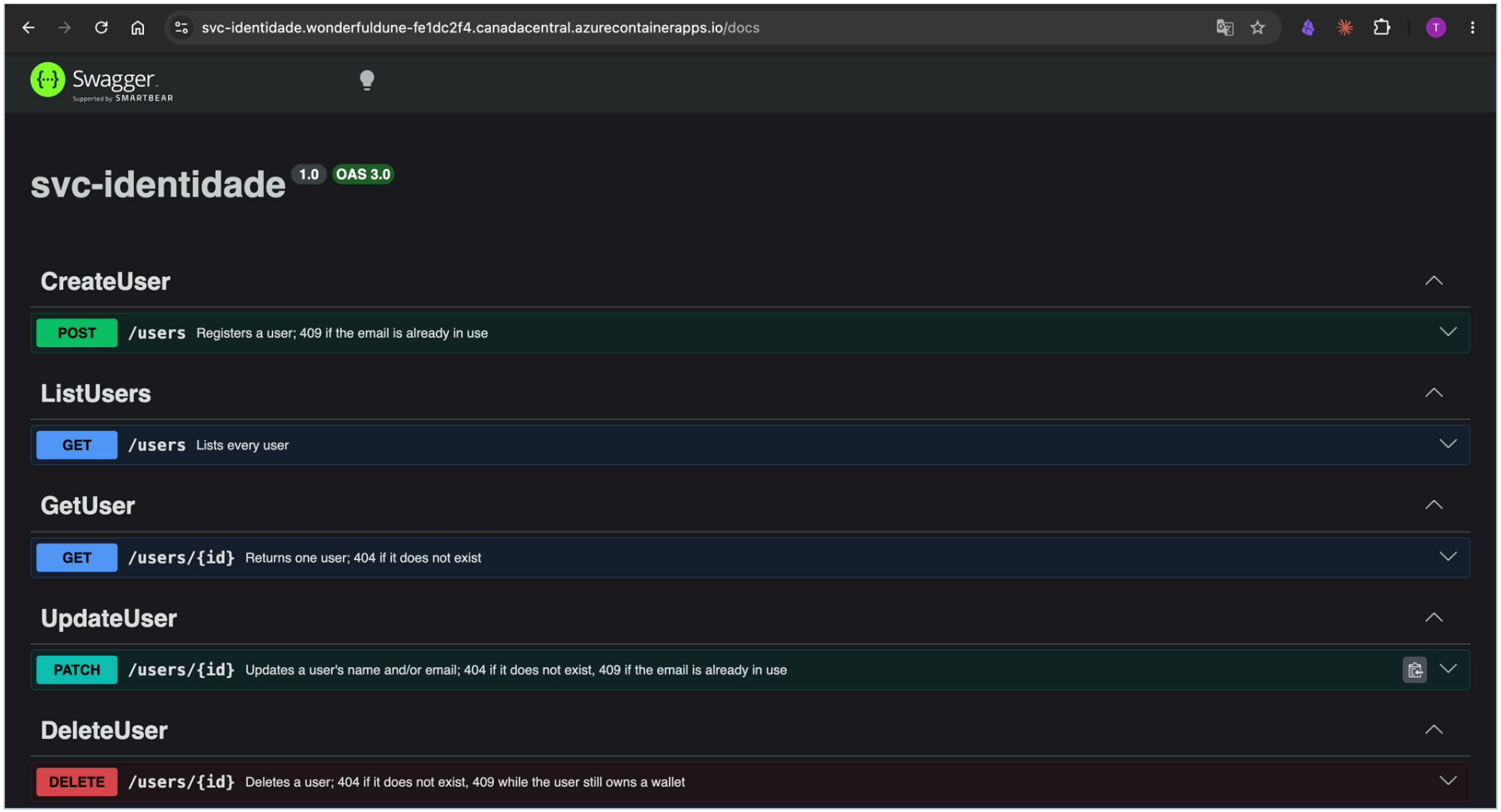


Print A.1 — Swagger do BFF pela URL do API Gateway

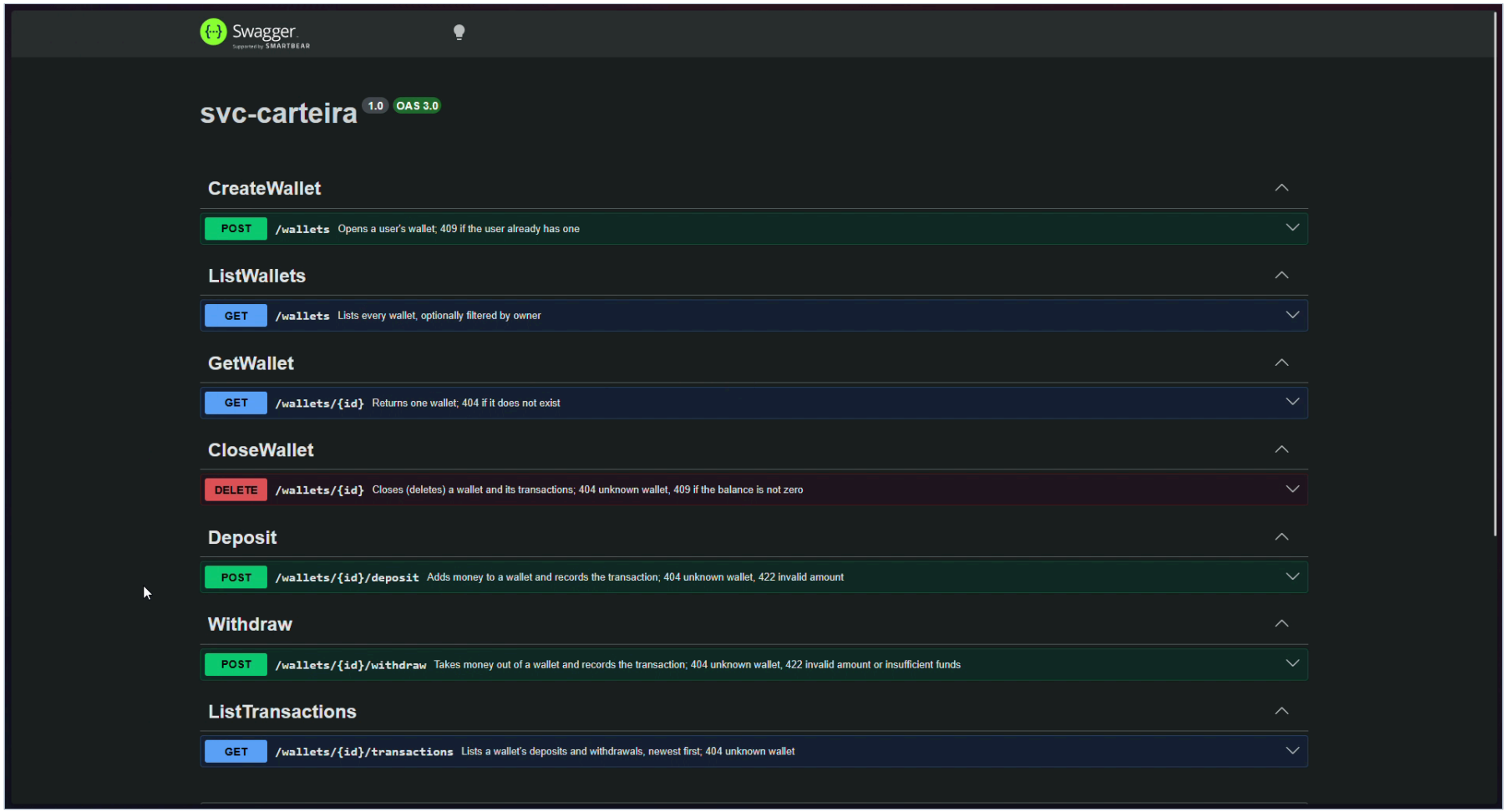
Swagger — BFF



Print A.2 — Todas as rotas do BFF no Swagger

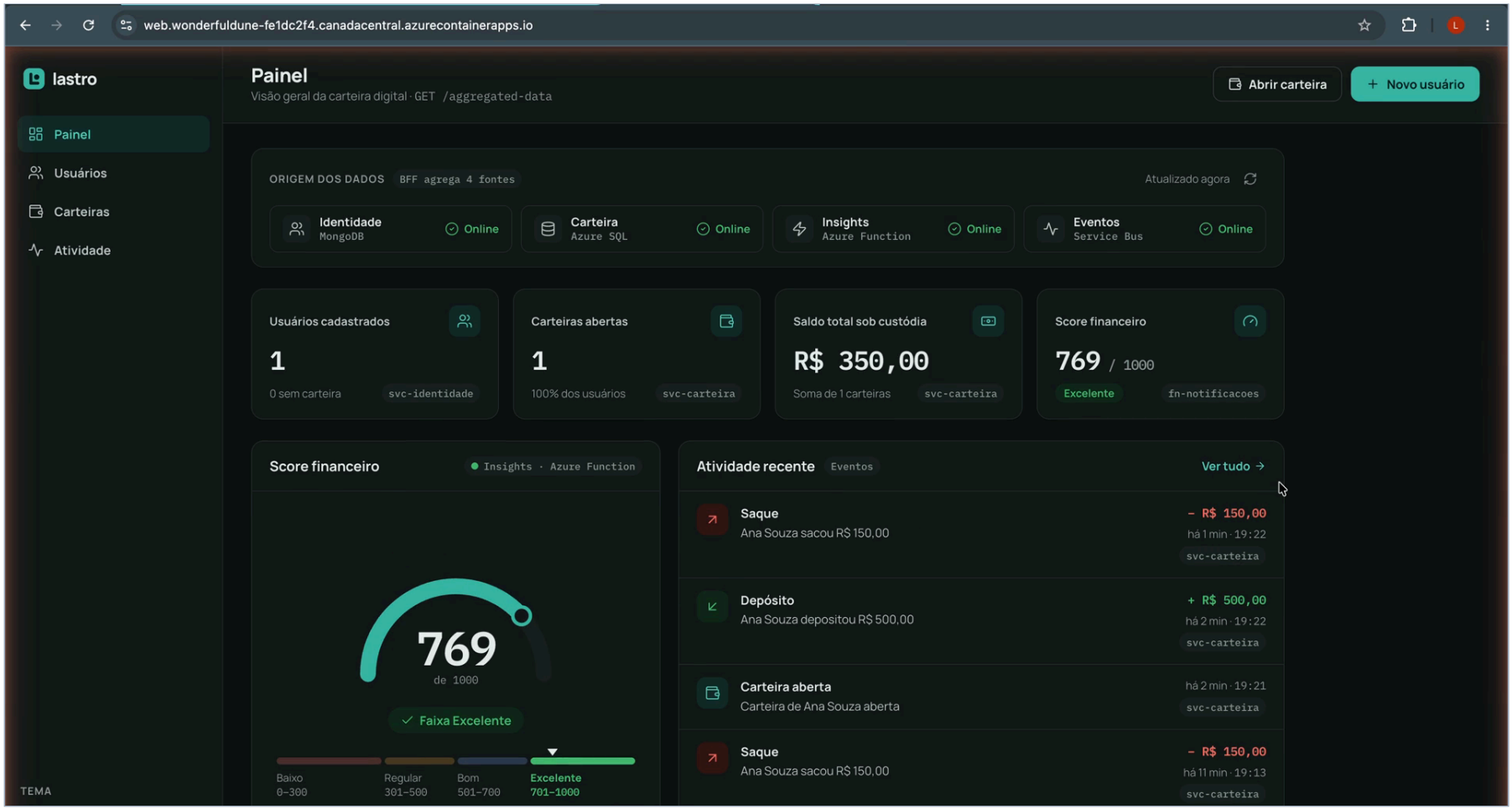


Print A.3 — Swagger do svc-identidade (MongoDB Atlas)



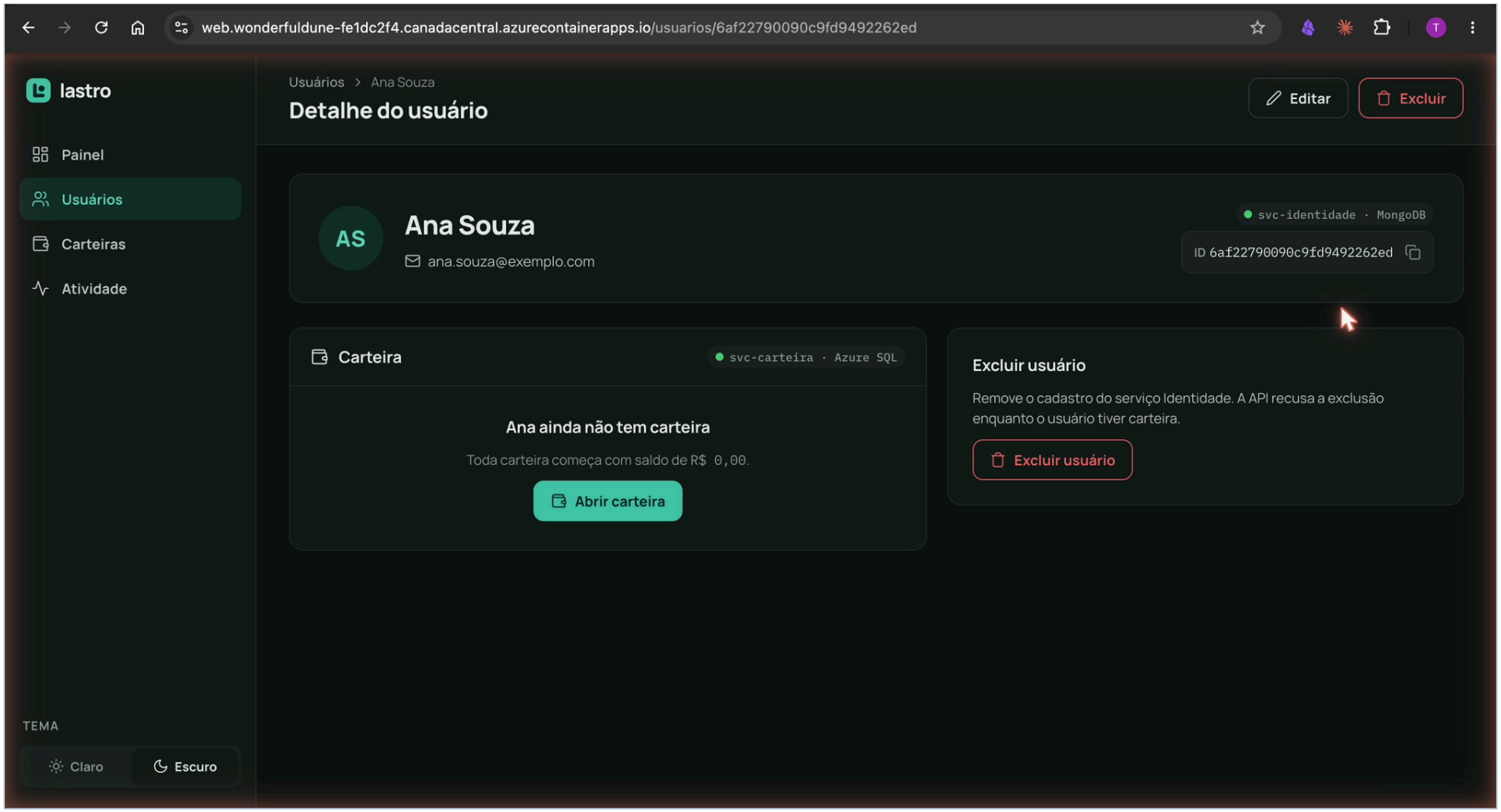
Print A.4 — Swagger do svc-carteira (Azure SQL)

Frontend funcionando



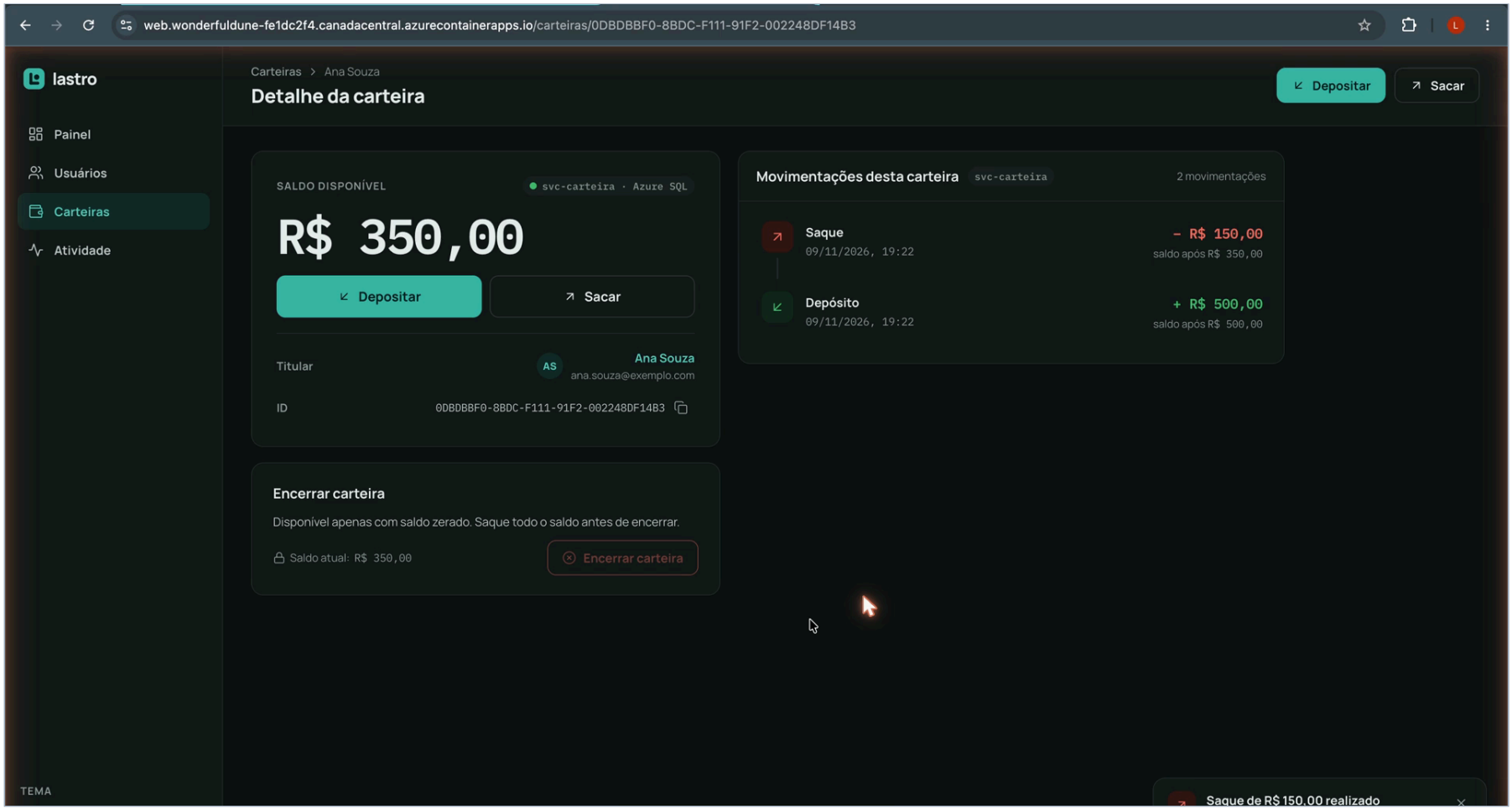
Print A.5 — Painel: usuários, carteiras e score (GET /aggregated-data) e atividade recente (GET /activity)

Frontend funcionando

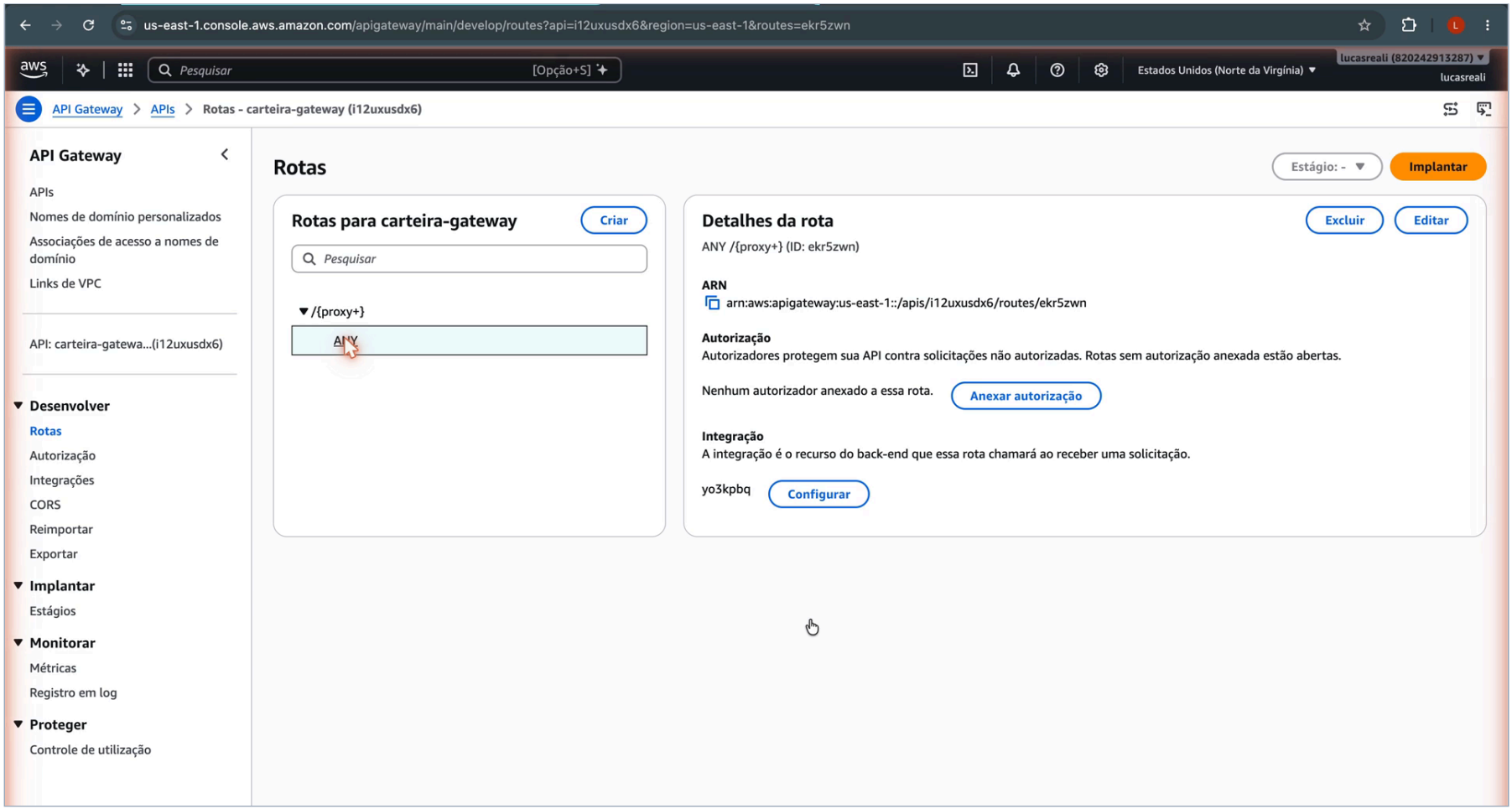


Print A.6 — Detalhe de usuário, com o ID gerado pelo MongoDB e o card de carteira vindo do svc-carteira

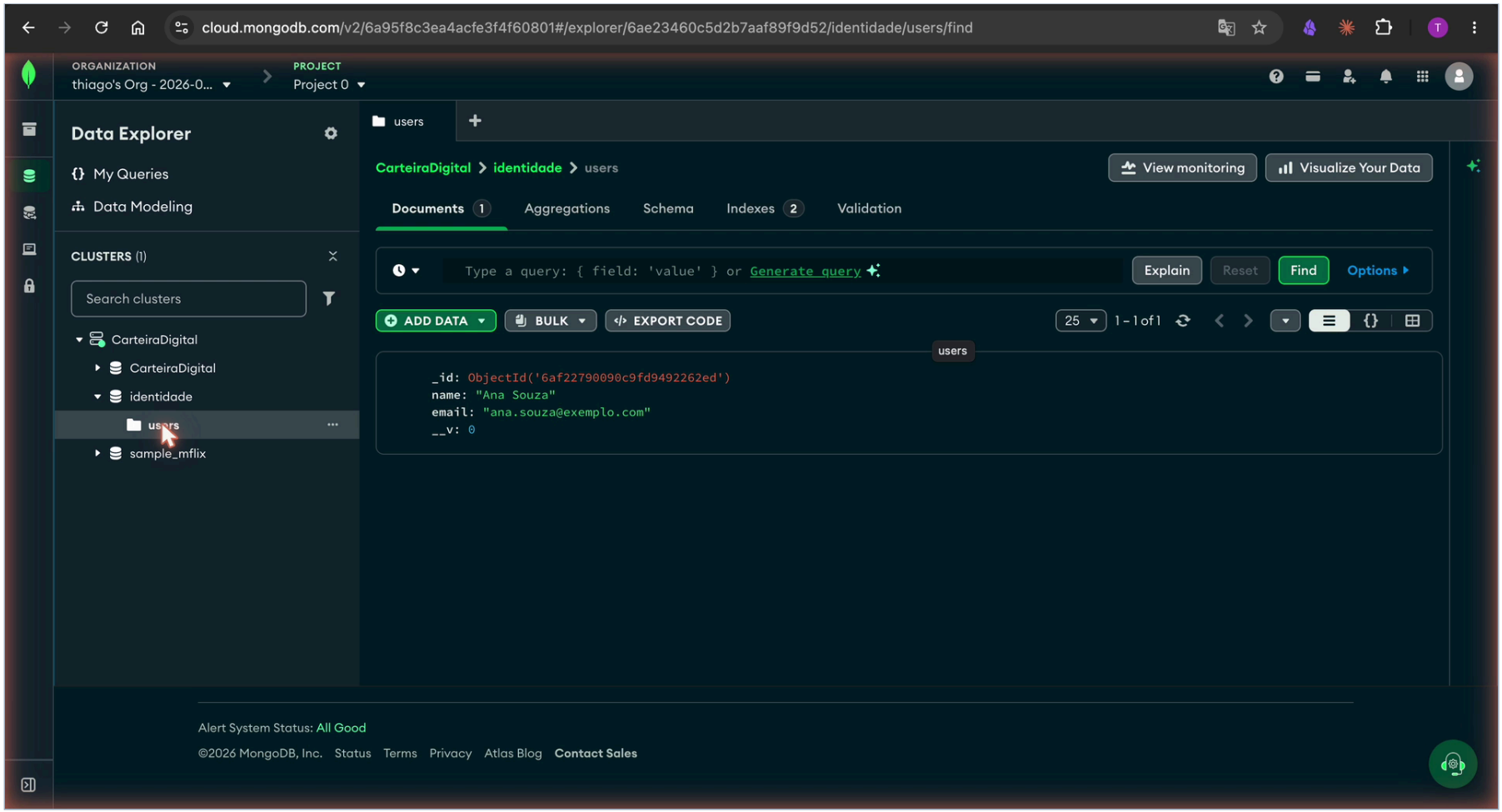
Frontend funcionando



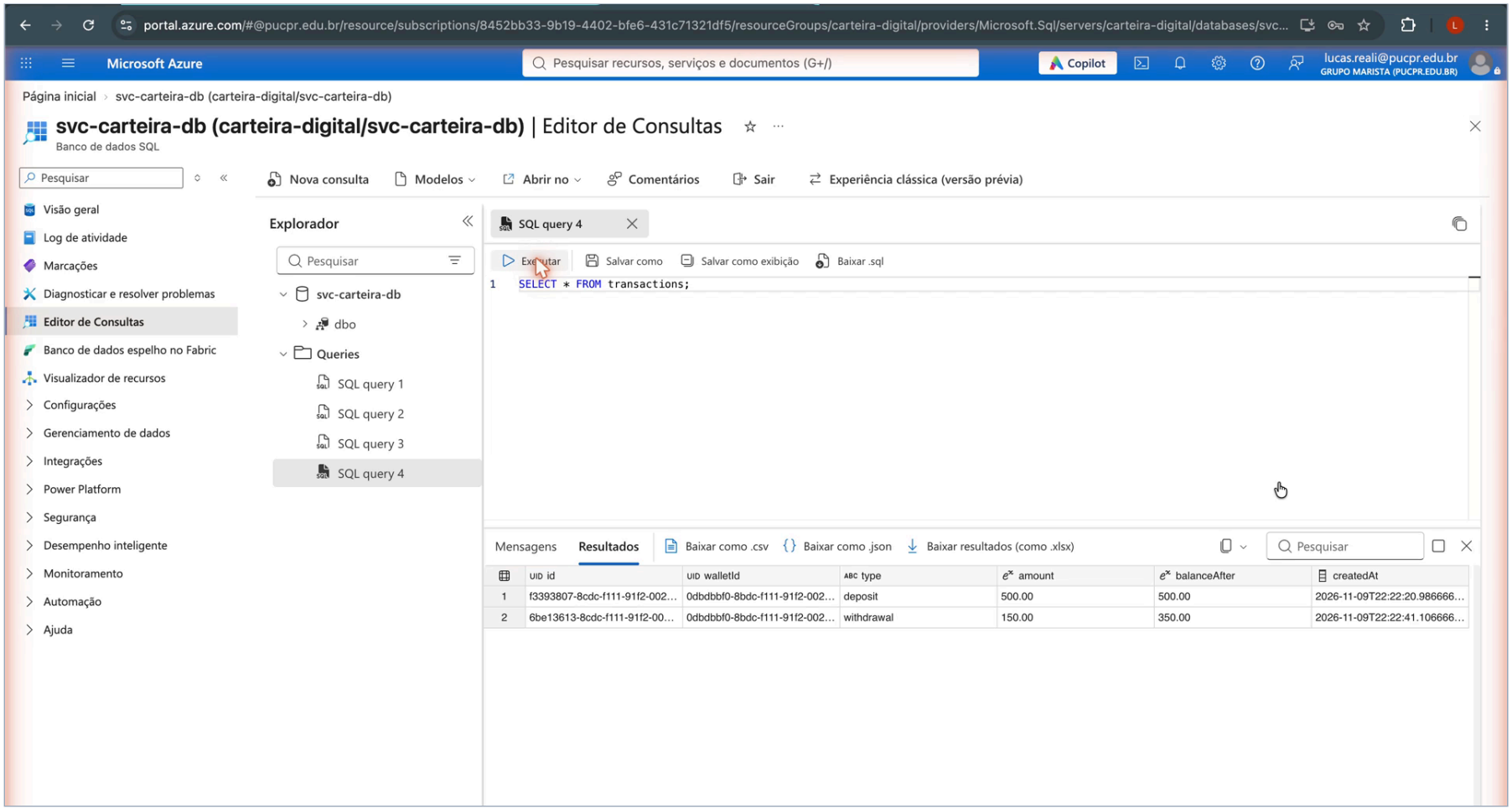
Print A.7 — Detalhe de carteira com saldo e extrato (depósito e saque)



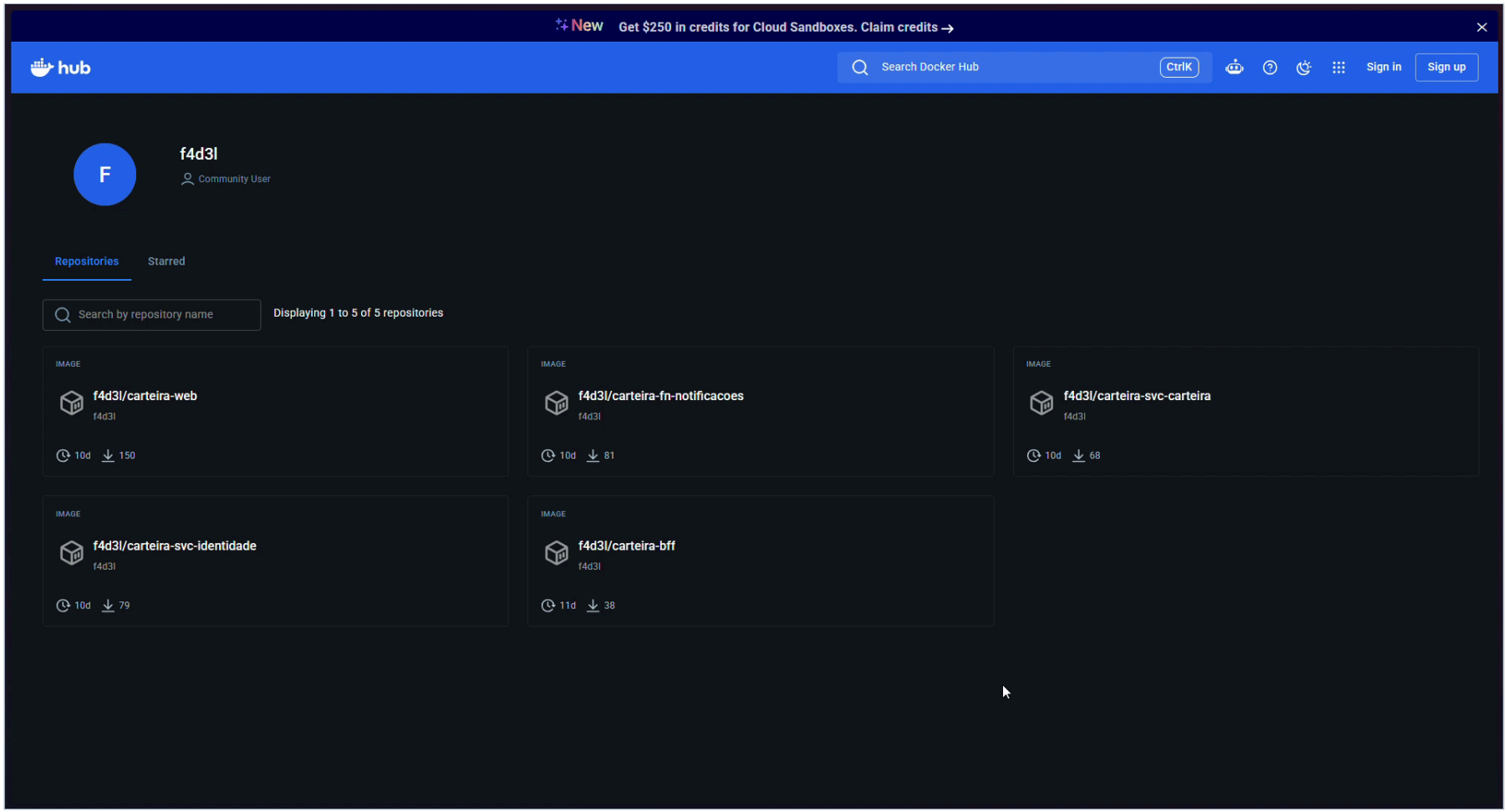
Print A.8 — AWS API Gateway: rotas do carteira-gateway e detalhes da rota ANY /{proxy+}



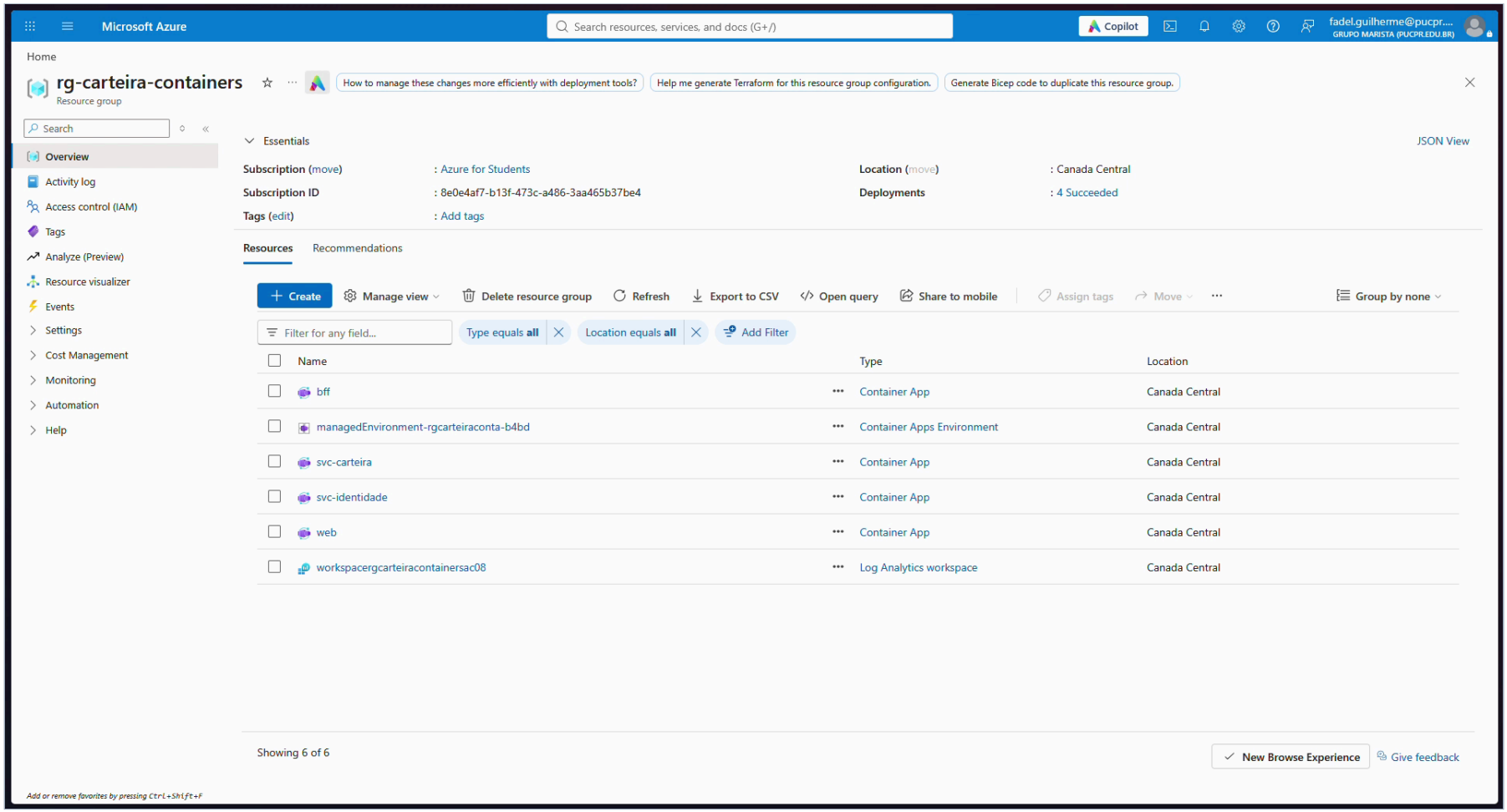
Print A.9 — MongoDB Atlas: coleção users do banco identidade



Print A.10 — Azure SQL: consulta à tabela transactions no editor de consultas



Print A.11 — Docker Hub: as cinco imagens publicadas em f4d3l



Print A.12 — Azure: grupo rg-carteira-containers com os Container Apps

Referências

- arc42 – Template de documentação de arquitetura, versão 8.2-PT. arc42.org
- Simon Brown – The C4 model for visualising software architecture. c4model.com
- Gernot Starke, Patrick Roos e colaboradores – Architecture Inception Canvas. canvas.arc42.org
- OMG – Unified Modeling Language, versão 2.5.1.
- Robert C. Martin – Clean Architecture. Prentice Hall, 2017.
- Jimmy Bogard – Vertical Slice Architecture.
- Documentação oficial: Azure Container Apps, Azure Functions, Azure Service Bus, Azure SQL Database, Azure Table Storage, AWS API Gateway (HTTP APIs) e MongoDB Atlas.
- Slides da disciplina Arquitetura de Soluções com Cloud (PUCPR) e enunciado do PJBL – Entrega 3.